B-BOT: A WiFi-Enabled Self-Balancing Wheel-Legged Robot with ROS 2 Vision Teleoperation

Third Year Individual Project – Final Report

May 2026

Botao Su

Your Identification Number: **1107124**

Supervisor: **Dr Liam Marsh**

Abstract

B-BOT is a self-balancing wheel-legged robot developed to investigate how real-time embedded balance control can be combined with safe wireless and vision-based teleoperation. Wheel-legged robots offer more mobility than conventional two-wheeled balancing robots because they can change body height and coordinate wheel and leg motion, but this also increases the coupling between body pitch, wheel torque and leg geometry. The main design constraint in this project was therefore that stabilising feedback must remain local to the embedded controller and must not depend on WiFi, camera processing or host-side software timing.

The implemented system uses an ESP32 controller running FreeRTOS tasks for motor feedback, inertial sensing, leg kinematics, target updating and balance control. The balance controller combines linear quadratic regulation for sagittal balance, PID loops for yaw, roll and leg-length regulation, and virtual model control to map virtual leg forces and torques to joint motor commands. A host-side ROS 2 vision bridge receives images from a ROS 2 WiFi camera module, runs MediaPipe-based perception, and sends supervisory commands to the robot through a WiFi TCP line protocol. Xbox BLE, UART commands, WiFi TCP commands and vision-generated commands are coordinated through shared command parsing, queue logic, watchdogs and safety gates.

The system was evaluated using control-loop jitter, WiFi command-entry latency, watchdog fault injection, camera bring-up, vision-command latency and live gesture-recognition tests. Comparative baseline testing was used to evaluate whether the additional controller complexity was justified. The measured control-loop log showed a mean period of 3.9998 ms and a median period of 4.000 ms over 15,000 samples. The WiFi TCP command-entry test achieved a median acknowledgement latency of 37.41 ms and p95 of 88.31 ms across 300 samples with zero unmatched commands. The watchdog fault-injection tests cleared direct commands in all 10 watchdog trials, detected all 10 TCP close trials, and produced full-stop events in all 10 idle trials. The live vision tests produced a clean 6/6 command-class matrix, 85.3% selected-frame label accuracy, and a median bridge-to-ESP32 acknowledgement latency of 66.13 ms across 71 safe commands. The main limitation is that the vision and WiFi layers are suitable only for supervisory teleoperation, not for closing the stabilising control loop. The project contributes a practical embedded wheel-legged balancing architecture extended with safety-aware multi-source command arbitration and ROS 2 vision teleoperation.

Declaration of Originality

I hereby confirm that this dissertation is my own original work unless referenced clearly to the contrary. Text, figures, code, data and ideas from other sources are acknowledged in the report, references or appendices as appropriate.

Intellectual Property Statement

The author of this thesis owns certain copyright or related rights in it. Third-party code, libraries and vendor materials used by the project are identified in the software appendix and references.

Contents

1 Introduction **1**

1.1 Background and motivation 1

1.2 Aims and objectives 2

1.3 Report structure 2

1.4 Project Management 3

2 Literature Review **4**

2.1 Introduction 4

2.2 From Two-wheeled Balancing to Wheel-legged Robots 4

2.3 Control Strategy Selection 5

2.4 Embedded Robot Software, ROS 2 and Vision Teleoperation 6

2.5 Gap Analysis and Design Implications 7

3 Methods **8**

3.1 Introduction — Top-level Requirements 8

3.2 System Architecture 9

3.3 Mechanical and Electrical Design 12

3.4 Perception and State Estimation 14

3.5 Control Architecture and Algorithms 15

3.6 ROS 2 Vision, WiFi TCP and Input Arbitration 17

3.7 Summary 19

4 Results and Discussion **20**

4.1 Evaluation Strategy 20

4.2 System Bring-up and Baseline 22

4.3 Balance and Motion Control Performance 25

4.4 WiFi TCP and ROS 2 Vision Teleoperation Performance 27

4.5 Summary 31

5 Conclusions and Future Work **32**

5.1 Conclusions 32
5.2 Future work 33

A Preliminary Project Proposal **37**

B Project Plan and Gantt Chart **37**

C Risk Register **39**

D Continuing Professional Development Log **39**

E Health and Safety Risk Assessment **40**

F Software Repository Link and README **40**

G Experiment Data, Logs and Extra Validation Evidence **41**

H Hardware and Control Modelling Evidence **42**

List of Figures

3.1 System architecture and data flow. 10
3.2 Firmware execution and communication timing. 11
3.3 Wheel-legged robot hardware layout. 12
3.4 Electrical and communication block diagram. 13
3.5 Hierarchical control block diagram. 17
3.6 Control state and safety behaviour diagram. 17
4.1 Control-loop timing distribution 24
4.2 TCP teleoperation command-entry step response 26
4.3 Vision event path from camera frame to ESP32 command acknowledgement. . 29
4.4 E10 live gesture confusion matrix 29
4.5 E11 vision bridge to ESP32 ACK latency 29
B.1 B-BOT reconstructed project Gantt chart. 38
H.1 Custom ESP32 controller PCB schematic export. 43
H.2 Custom ESP32 controller PCB layout export. 43

List of Tables

2.1 Control strategy comparison for B-BOT. 6
2.2 Literature gap to B-BOT design choice. 7
3.1 Top-level implementation requirements. 9
3.2 Firmware task timing and behaviour. 11
3.3 Main hardware components. 13
3.4 State variables used by the embedded controller. 14
3.5 Auxiliary PID loops. 16
3.6 Vision bridge operating modes. 18
3.7 Command source arbitration summary. 19
4.1 Evaluation matrix. 20
4.2 Baseline and comparison conditions. 21
4.3 Bring-up checklist. 22

4.4 Static balance metrics. 23
4.5 Control-loop timing jitter. 24
4.6 Impulse recovery metrics. 25
4.7 Leg-length sensitivity. 25
4.8 E4a TCP teleoperation command-entry step response. 26
4.9 E4b physical teleoperation response metrics. 27
4.10 E9 controller ablation metrics. 27
4.11 WiFi TCP command-entry latency. 27
4.12 Watchdog and disconnect response. 28
4.13 Camera and vision bridge metrics. 30
4.14 Command arbitration tests. 31
4.15 Objective, evidence and limitation summary. 32
5.1 Objective closure summary. 33
A.1 Proposal-to-final-project alignment. 37
B.1 Project-management phase summary. 38
C.1 Project risk register summary. 39
D.1 CPD activity summary. 40
E.1 Health and safety risk assessment summary. 40
F.1 Software repository evidence summary. 41
G.1 Experiment data status summary. 41
H.1 Hardware and control evidence summary. 42

1 Introduction

1.1 Background and motivation

Self-balancing robots are useful engineering platforms because they combine unstable dynamics, embedded sensing, real-time control and actuator coordination in a compact system. A conventional two-wheeled balancing robot mainly regulates body pitch while moving through wheel torque. A wheel-legged robot extends this concept by adding controllable leg geometry, allowing the body height, support force and dynamic posture to be changed. This improves mobility but also makes the control problem harder because body pitch, wheel motion, leg length and joint torque become strongly coupled.

This project developed B-BOT, a WiFi-enabled self-balancing wheel-legged robot based on an ESP32 embedded controller. The main engineering problem was not simply to make the robot stand. The harder problem was to integrate real-time embedded balance control with multiple non-real-time command sources without allowing those sources to destabilise the robot. B-BOT can receive input from an Xbox BLE controller, UART scripted commands, WiFi TCP commands from a host computer, and a ROS 2 / MediaPipe vision bridge. These inputs all affect the same physical machine, so the system must avoid conflicting commands and must fail safely when a command source disappears.

The central design decision was to keep the stabilising control loop local to the ESP32. Pitch, wheel and leg feedback are processed on the embedded controller, and the LQR/PID/VMC control computation is executed in FreeRTOS tasks. Host-side software is deliberately kept outside the balance loop. This is necessary because WiFi latency, camera frame rate, ROS 2 scheduling and MediaPipe inference are not deterministic at the 4 ms timescale targeted by the balance controller.

The ROS 2 part of the project is therefore used as a supervisory perception layer rather than as the low-level control bus. The ROS 2 WiFi camera module provides images to the host ROS 2 system through its module-side micro-ROS image transport. The host-side `wheeleg_vision_bridge` processes those images and converts recognised visual events into text commands. Those commands are then sent to the ESP32 over a WiFi TCP line protocol. In the current implementation, the bottom ESP32 robot controller is not a micro-ROS node.

The motivation for this architecture is practical safety. Vision teleoperation is useful for demonstration and human interaction, but an incorrect gesture classification or network delay should not directly destabilise the robot. By treating vision and WiFi commands as temporary target requests with watchdog expiry, B-BOT combines local real-time balance control with higher-level remote inputs in a controlled and testable way. The project contribution is therefore an integrated safety-aware embedded robot architecture, not a claim that vision or WiFi are suitable for closing the stabilising feedback loop.

1.2 Aims and objectives

The aim of the project was to design, implement and evaluate a self-balancing wheel-legged robot that combines real-time embedded balance control with safe wireless and vision-based supervisory teleoperation.

The project objectives were:

O1 — Embedded wheel-legged balance control. Develop an ESP32-based control system that combines LQR, PID loops and VMC to stabilise the robot, regulate leg geometry and command wheel/joint torques. This objective is evaluated through static balance, disturbance recovery, leg-length variation, teleoperation step response and control-loop jitter tests.

O2 — ROS 2 vision input and WiFi TCP command channel. Implement a host-side ROS 2 vision pipeline and WiFi TCP command path that can generate and transmit supervisory robot commands without entering the stabilising feedback loop. This objective is evaluated through TCP command-entry latency, camera topic checks, vision bridge ACK latency and live gesture-recognition tests.

O3 — Safe multi-source input arbitration. Support Xbox BLE, UART scripted commands, WiFi TCP commands and vision-generated commands without allowing uncontrolled conflicts between input sources. This objective is evaluated through command queue behaviour, direct-command watchdogs, TCP disconnect/idle fault injection, `dry_run` vision tests, stunt arming behaviour and stop-command validation.

The success criteria are not based on maximum speed or fully autonomous navigation. Instead, the project is judged by whether the implemented robot demonstrates stable embedded balancing, predictable command handling and evidence-based safety behaviour under realistic communication and perception failures.

1.3 Report structure

Chapter 2 reviews the relevant literature on two-wheeled and wheel-legged balancing robots, control strategies for unstable mobile robots, and ROS 2 / micro-ROS communication in non-deterministic networks. It is used to identify the gap that motivates the selected architecture. Chapter 3 presents the implemented system design, including hardware integration, embedded task structure, sensing, state estimation, control algorithms, WiFi TCP communication and input arbitration. Chapter 4 evaluates the system using experiments mapped directly to the three objectives. Chapter 5 concludes the report by summarising the technical contribution, identifying limitations and proposing future work.

1.4 Project Management

The project was managed as an iterative engineering build rather than as a single linear implementation. The initial plan separated the work into hardware bring-up, embedded control, communication integration, host-side vision, testing and final reporting. Appendix B contains the project plan and Gantt chart, including the final schedule used during the report-writing phase. The main deviation from the original plan was that low-level balance control and safety handling required more time than expected, while the ROS 2 vision layer had to be scoped as a supervisory teleoperation feature rather than as part of the stabilising control loop.

Risk management was treated in two parts. First, the Health and Safety Risk Assessment in Appendix E covers risks to the student and other people during practical work, including high-torque moving joints, a falling self-balancing robot, soldering burns and fumes, battery and wiring faults, sharp printed or machined parts, trailing cables, and unexpected motion during wireless or vision testing. These hazards were managed through supervised/lab-appropriate working, low-energy bench tests before free-standing tests, keeping hands clear of joints and wheels, using appropriate soldering practice and ventilation, securing the robot during software tests, and keeping power isolation available.

Second, the Risk Register in Appendix C covers risks to successful project delivery, such as hardware availability, motor wiring reliability, IMU calibration errors, unstable tuning, loss of experiment data and integration delays. These project risks influenced the technical design: balance-disabled states produce zero torque, direct commands expire through watchdogs, WiFi disconnects inject a full-stop command sequence, and the vision bridge defaults to `dry_run`. This separation avoids treating safety paperwork as a purely technical design list while still showing how safety and delivery risks shaped the project.

The project also required continuing professional development across embedded control, FreeRTOS scheduling, ROS 2, micro-ROS camera integration, MediaPipe and experimental data analysis. Appendix D contains the CPD log. This self-directed learning affected the final architecture: instead of attempting to place the whole robot inside ROS 2, the design separates the hard real-time ESP32 controller from the host-side perception pipeline. This was a practical project-management decision as well as a technical one, because it reduced integration risk while preserving a clear demonstration of vision-based teleoperation.

2 Literature Review

2.1 Introduction

This literature review is organised by design problem rather than by individual paper. The project combines three areas: self-balancing wheel-legged robotics, embedded control of coupled unstable systems, and remote robot command interfaces over non-deterministic communication links. Reviewing these areas separately makes it possible to justify the architecture used in this project: real-time stabilisation remains on the ESP32, while ROS 2 vision and WiFi communication are used only as supervisory command inputs.

Section 2.2 reviews the development from two-wheeled inverted-pendulum robots to wheel-legged robots. Section 2.3 compares the main control approaches relevant to this project, including PID, LQR, ADRC and VMC. Section 2.4 reviews ROS 2, micro-ROS and lossy-network communication from the perspective of safe teleoperation. Section 2.5 then summarises the technical gaps that motivate the design choices presented in Chapter 3.

2.2 From Two-wheeled Balancing to Wheel-legged Robots

Two-wheeled self-balancing robots are commonly modelled as mobile inverted pendulums. A representative early example is JOE, which demonstrated that a compact two-wheeled robot can balance and move using feedback control around an unstable upright equilibrium [1]. Later reviews show that this class of robot is attractive because the mechanical structure is simple, but the control problem is non-trivial: the robot must regulate body pitch while also responding to wheel motion, actuator limits and external disturbances [2].

Wheel-legged robots extend this problem by adding controllable leg geometry. This increases capability because the robot can change body height, absorb impacts, step over small obstacles or perform dynamic actions. The cost is that the balance dynamics are no longer fixed. Leg length changes the centre of mass and the effective inverted-pendulum geometry, while joint motion couples support force, body pitch and wheel torque. For this reason, a wheel-legged robot cannot be treated as only a two-wheeled inverted pendulum with extra actuators.

Ascento is a useful reference point because it demonstrates the mobility benefit of a two-wheeled jumping wheel-legged platform [3]. Its relevance to this project is not that B-BOT has the same actuator power or dynamic performance, but that it shows why wheel and leg actuation must be coordinated rather than controlled independently. More recent work on wheel-legged balancing has investigated combined control structures, including LQR with disturbance rejection and unified control frameworks for model variation [4], [5]. These studies support the idea that a wheel-legged controller should be evaluated under

disturbance and changing leg configuration, not only in a static standing pose.

The implication for B-BOT is a specific trade-off. The platform is a small ESP32-based robot rather than a high-power research platform, so the controller must be computationally light and robust enough for repeated demonstration. At the same time, the wheel-legged mechanism still requires leg kinematics, leg-length-dependent behaviour and measured recovery performance. This motivates the E2 disturbance recovery test, the E3 leg-length sensitivity test and the E9 controller ablation used later in the report.

2.3 Control Strategy Selection

The relevant control methods can be compared by asking what part of the system they control well and what assumptions they introduce. PID control is attractive for embedded implementation because it is simple, computationally cheap and easy to tune on individual variables. It is suitable for local objectives such as yaw correction, roll correction and leg-length regulation. Its weakness is that independent PID loops do not naturally capture the coupled relationship between body pitch, wheel position, wheel velocity and leg geometry.

LQR is more appropriate for the sagittal balancing problem because it controls a coupled state vector through a single feedback gain. Dynamic models of two-wheeled inverted-pendulum robots show why body pitch, pitch rate, wheel position and wheel velocity should be considered together rather than as independent loops [6]. For B-BOT, this makes LQR a suitable core controller for pitch and wheel balance. The limitation is that LQR quality depends on the model, operating point and gain tuning. A fixed gain may work near one leg length, while a wheel-legged platform benefits from scheduling the gain by virtual leg length.

ADRC addresses disturbance and model uncertainty by estimating and rejecting the total disturbance acting on a system [7]. This is attractive for wheel-legged robots because contact changes, modelling errors and external pushes are difficult to model exactly. However, ADRC introduces observer dynamics and additional tuning parameters. In this project, the implementation risk and tuning time were higher than the expected benefit for the available hardware and schedule. ADRC is therefore used as a literature comparator rather than as the selected embedded controller.

VMC provides a complementary abstraction. Instead of commanding each joint directly, the controller defines virtual forces and torques acting on the robot body or leg, then maps those virtual quantities to actuator torques [8]. This is useful for B-BOT because virtual leg force and hip torque are more meaningful for balancing than individual joint torques. In the implemented design, LQR and PID determine the desired whole-body effects, and VMC maps those effects to the two joint motors on each side.

Table 2.1. Control strategy comparison for B-BOT.

Strategy	Strength	Limitation	Role in this project	Evaluation link
PID	Simple, fast, easy to tune locally	Weak for strongly coupled whole-body dynamics	Yaw, roll, leg length and auxiliary regulation	E1/E4b stability and response
LQR	Handles coupled state feedback in one controller	Depends on model quality and operating point	Main sagittal balance controller	E2 disturbance recovery
Gain-scheduled LQR	Adapts feedback to changing leg length	Requires leg-length estimate and scheduled gains	Adjusts balance behaviour as body height changes	E3 leg-length sensitivity and E9 ablation
ADRC	Robust to unmodelled disturbances	More observer and tuning complexity	Literature comparator, not implemented	Used for discussion, not direct claim
VMC	Maps meaningful virtual forces to joint torques	Requires kinematic mapping and force limits	Converts support force and hip torque to joint torques	E2/E3 recovery behaviour

The selected controller is therefore hybrid rather than method-pure. LQR is used where coupling is strongest, PID is used where local regulation is sufficient, and VMC bridges the gap between whole-body virtual forces and actuator-level torques. This is a pragmatic engineering choice: it is less theoretically complete than a full nonlinear whole-body controller, but it is feasible on the ESP32 and produces testable design claims.

2.4 Embedded Robot Software, ROS 2 and Vision Teleoperation

ROS 2 is widely used for robot software because it provides modular communication, standard message types, launch tooling and integration with perception libraries [9]. Its middleware design includes quality-of-service policies that allow developers to choose trade-offs such as reliability, history depth and durability for different data streams [10]. These features are useful for camera images, status topics and host-side processing, but they do not remove the real-time requirements of a balancing controller.

micro-ROS extends ROS 2 concepts to resource-constrained embedded devices through the DDS-XRCE model [11], [12]. In a typical micro-ROS system, a microcontroller communicates with a host-side agent, and the agent bridges between DDS-XRCE and the ROS 2 graph. eProsima's Micro XRCE-DDS documentation describes the transport-layer options used by such systems [13]. This architecture is valuable when a microcontroller needs to publish or subscribe to ROS 2 topics directly, but it is not automatically the best choice for a time-critical balancing loop on a small platform.

B-BOT uses a narrower and safer boundary. The bottom ESP32 controller is not a micro-ROS node. Instead, micro-ROS is used only in the ROS 2 WiFi camera module image path: the camera module publishes images to the host ROS 2 system, and the host-side vision bridge processes those images. Robot commands are then transmitted to the ESP32 over a WiFi TCP line protocol. This distinction is important because it avoids overstating the role of ROS 2 in the balance-critical path and keeps the stabilising loop dependent only on local IMU and

motor feedback.

MediaPipe is relevant to the host-side perception layer because it provides a graph-based framework for real-time perception pipelines [14]. The hand-gesture component in B-BOT is aligned with MediaPipe Hands, which was designed for real-time hand landmark tracking [15]. These references justify MediaPipe as a practical perception tool, but they do not prove reliable robot control in B-BOT's specific camera, lighting and gesture conditions. This is why the report includes E10 as a live gesture confusion matrix rather than treating vision recognition as a demonstrated fact.

Wireless command links introduce failure modes that are not present in a local feedback loop. Packets can be delayed, TCP clients can disconnect, the host process can crash, and a vision algorithm can output stale or incorrect events. For a self-balancing robot, these failures cannot be allowed to hold a non-zero command indefinitely. The literature supports modular distributed robot software, but the safety requirement in this project is more direct: remote commands must be temporary target requests with watchdog expiry, not hard real-time control signals.

This motivates the command architecture used in Chapter 3. WiFi TCP and vision commands update target variables or enqueue scripted actions, while the local ESP32 balance loop continues to use local IMU and motor feedback. Watchdogs, disconnect handling, `dry_run` mode and explicit stunt arming are therefore not secondary features. They are required to make remote and vision-based control acceptable for a physically unstable robot.

2.5 Gap Analysis and Design Implications

The literature shows that B-BOT sits between two established areas. From balancing-robot work, it inherits the need for fast local feedback around an unstable body. From wheel-legged robot work, it inherits the need to consider changing leg geometry and support forces. From ROS 2, micro-ROS and MediaPipe work, it inherits the opportunity to use modular host-side perception and communication. The main engineering challenge is to combine these areas without allowing non-deterministic host-side processing to destabilise the robot.

Table 2.2. Literature gap to B-BOT design choice.

Gap identified from literature	Design choice in B-BOT	Evidence used later
Wheel-legged robots have coupled pitch, wheel and leg dynamics	Use LQR for sagittal balance, PID for auxiliary loops and VMC for joint torque mapping	E2 disturbance recovery
Leg length changes the operating point	Schedule the LQR gain using virtual leg length	E3 leg-length sensitivity and E9 ablation

Gap identified from literature	Design choice in B-BOT	Evidence used later
Embedded hardware has limited computation and timing margin	Keep the balance loop local to ESP32 FreeRTOS tasks	E8 control-loop jitter
ROS 2 and MediaPipe are useful but non-deterministic relative to a 4 ms control loop	Treat ROS 2 vision and WiFi TCP as supervisory command inputs only	E5/E11 latency and E10 confusion matrix
Remote commands can become stale after host or network failure	Add direct-command watchdogs and TCP idle/disconnect full-stop	E6 fault injection
Vision-generated actions can be unsafe if misclassified	Default the bridge to `dry_run` and block high-risk actions unless armed	E10 audit failures and stunt gate test

This gap analysis defines the Methods chapter. The report does not claim that B-BOT solves general autonomous wheel-legged locomotion or that vision is suitable for stabilising feedback. Its contribution is narrower and more defensible: a practical embedded wheel-legged balancing system with a safety-aware wireless and vision teleoperation layer, evaluated through timing, safety and perception-reliability experiments.

3 Methods

3.1 Introduction — Top-level Requirements

The system was designed around one primary constraint: the balance loop must remain local to the ESP32 controller and must not depend on WiFi, ROS 2, camera processing, or any host-side software. Wireless communication and vision processing were therefore treated as supervisory command inputs rather than components of the stabilising feedback loop. This separation reduces the risk that variable WiFi latency, camera frame-rate variation, or MediaPipe inference delay can directly destabilise the robot.

The top-level requirements used to guide the implementation are summarised in Table 3.1. The timing values are implementation targets from the firmware task structure and are later checked experimentally where possible. The table is also used to keep the Methods and Results chapters aligned: the 4 ms control period is evaluated by E8, the WiFi watchdogs by E6, and the vision safety defaults by E10 and E11.

Table 3.1. Top-level implementation requirements.

Requirement area	Target / implementation choice	Rationale
Balance control locality	All balance-critical computation runs on the ESP32	Prevents wireless or host-side delays from entering the stabilising loop
Main control period	4 ms target period for `CtrlBasic_Task`	Provides a sufficiently fast update rate for pitch and wheel-leg stabilisation
Leg kinematics update	4 ms target period	Keeps leg length and leg angle estimates aligned with the control loop
IMU update	5 ms target period using MPU6050 DMP packets	Provides attitude and angular-rate feedback at a comparable rate to the control loop
CAN motor feedback/output	2 ms receive/send polling in firmware tasks	Maintains fresh wheel and joint motor state for the controller
Manual control input	Xbox BLE processing at approximately 50 Hz	Gives responsive human teleoperation without entering the hard real-time path
WiFi direct-command safety	500 ms watchdog for `DRIVE` and `YAWRATE` direct commands	Prevents stale remote commands from continuing after host or network failure
TCP client safety	1500 ms idle watchdog plus full-stop on client drop / WiFi loss	Ensures the robot is commanded to stop when the TCP command stream fails
Vision safety	`dry_run` default enabled; `stunt_armed` default disabled	Prevents unverified vision outputs from driving the robot or triggering high-risk actions

These requirements define the structure of this chapter. Section 3.2 introduces the overall architecture and task timing. Section 3.3 describes the mechanical and electrical integration. Section 3.4 explains sensing and state estimation. Section 3.5 presents the LQR/PID/VMC control design. Section 3.6 describes the ROS 2 vision bridge, WiFi TCP command interface, and input arbitration logic.

3.2 System Architecture

The implemented system uses the layered architecture shown in Fig. 3.1. The lowest layer consists of the physical robot: the ESP32 controller, MPU6050 inertial measurement unit (IMU), CAN-connected wheel and joint motors, battery monitoring circuitry, and the wheel-leg mechanism. The middle layer is the ESP32 firmware, which performs motor feedback processing, leg kinematics, state estimation, balance control, command parsing, and safety handling. The upper layer consists of optional command sources: Xbox BLE manual

control, UART2 scripted commands, WiFi TCP commands from a PC tool, and ROS 2 / MediaPipe vision commands generated on the host computer.

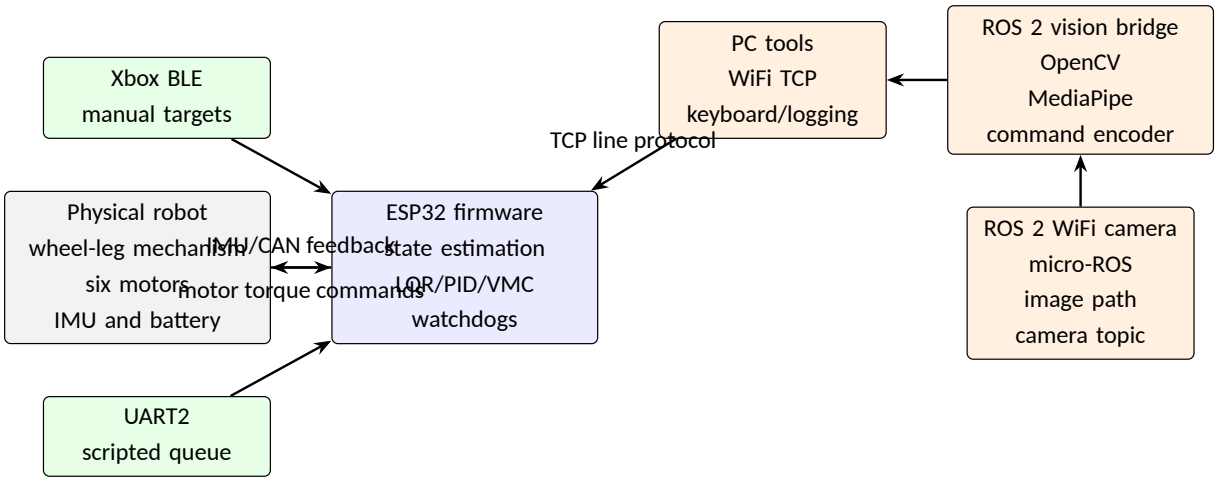

Fig. 3.1. System architecture and data flow.

The key architectural decision is that the ESP32 is not used as a micro-ROS node in the current implementation. Instead, micro-ROS is used only in the camera-to-host image path: the ROS 2 WiFi camera module publishes compressed image messages to the ROS 2 host via a micro-ROS agent. The host-side `wheeleg_vision_bridge` subscribes to the camera topic, runs MediaPipe-based perception, encodes the resulting visual event into the same textual command protocol used by other tools, and sends the command to the robot over WiFi TCP. The robot therefore receives vision commands through the same parser as keyboard or scripted commands.

This architecture keeps the fast balance loop isolated. The ESP32 control loop uses local IMU, wheel, and leg states. Host-side vision can request a target velocity, yaw rate, jump, or other high-level action, but it cannot directly close the balance loop. This is important because the camera frame rate, ROS 2 scheduling, MediaPipe inference time, and WiFi latency are all non-deterministic relative to the 4 ms control period.

Fig. 3.2 shows the firmware execution model. The most time-critical tasks are the CAN feedback/output tasks, the leg kinematics task, the IMU task, the target update task, and the main control task. Lower-priority tasks handle BLE input, UART2 commands, WiFi TCP command reception, motor calibration, and diagnostic logging.

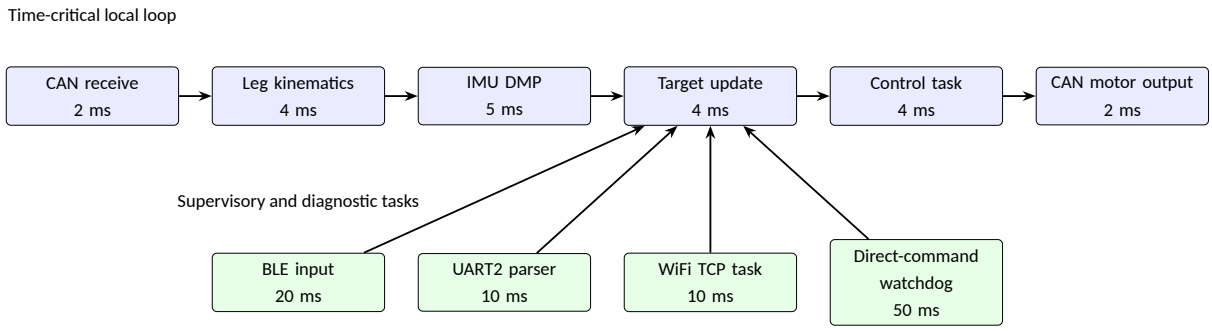

Fig. 3.2. Firmware execution and communication timing.

Table 3.2. Firmware task timing and behaviour.

Firmware component	Main function	Nominal timing / behaviour
`CAN_RecvTask`	Polls CAN motor feedback	2 ms polling
`Motor_SendTask`	Sends motor voltage commands over CAN	2 ms loop delay
`IMU_Task`	Reads MPU6050 DMP packet and gyro data	5 ms target period
`LegPos_UpdateTask`	Computes virtual leg length, angle and derivatives	4 ms target period
`Ctrl_TargetUpdateTask`	Applies speed ramping, position target limiting and yaw target integration	4 ms target period
`CtrlBasic_Task`	Computes LQR/PID/VMC control output	4 ms target period
`BLE_TestTask`	Processes Xbox controller notifications	20 ms target period
`UART2_CommandTask`	Parses UART2 command lines	10 ms target period
`WiFiCmdTask`	Accepts TCP client, parses command lines, handles disconnects	10 ms loop delay when connected
`Serial_Task`	Applies direct-command watchdogs	50 ms loop delay

The data flow inside the firmware is deliberately simple. Sensor and motor tasks update global state variables. The target update task smooths external target commands. The control task reads the current state and target values, calculates wheel and joint torque commands, and writes them to the motor abstraction. Command sources do not drive actuators directly; they only update target values or enqueue scripted actions. This separation is what makes the communication experiments in Chapter 4 meaningful: a delayed WiFi command can change a target, but it cannot replace the local feedback path.

3.3 Mechanical and Electrical Design

The robot platform was designed and built as part of this project rather than being a vendor robot base. The mechanical layout consists of two wheel modules and two articulated legs. Each side has one wheel motor and two leg joint motors, giving six motors in total: four joint motors for changing the virtual leg geometry and two wheel motors for ground contact and balance control. The Yahboom component used in the system is the separate ROS 2 WiFi camera module in the vision path.

The ESP32 was selected as the embedded controller because it provides sufficient processing capability for the control loop while also supporting Bluetooth Low Energy (BLE), WiFi, FreeRTOS tasks, and Arduino/PlatformIO development [16]–[18]. The controller electronics use a custom ESP32 PCB designed for this robot. The board integrates the IMU path, six motor interfaces, an additional expansion interface and power-management circuitry. PCB schematic and layout exports are included in Appendix H as hardware-design evidence. The firmware is built using PlatformIO with the `esp32doit-devkit-v1` environment as a compatible ESP32 Arduino build target; this is a toolchain target selection, not a claim that a commercial ESP32 development board is the robot controller.

The MPU6050 IMU provides body attitude and angular-rate feedback. The motor controllers communicate with the ESP32 over CAN, allowing joint and wheel angle/speed feedback to be used by the leg kinematics and balance controller. Battery voltage is monitored through an ADC path, allowing the firmware to observe the supply condition and support future output compensation or low-voltage safety logic.

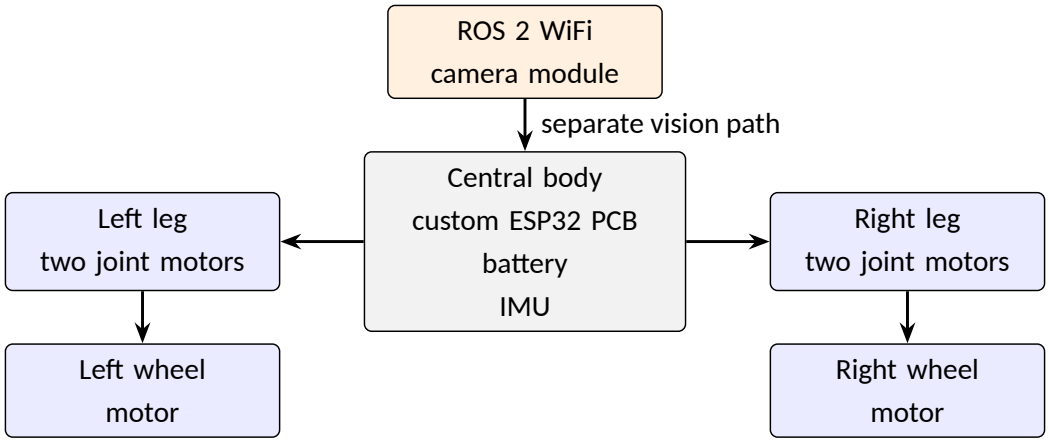

Fig. 3.3. Wheel-legged robot hardware layout.

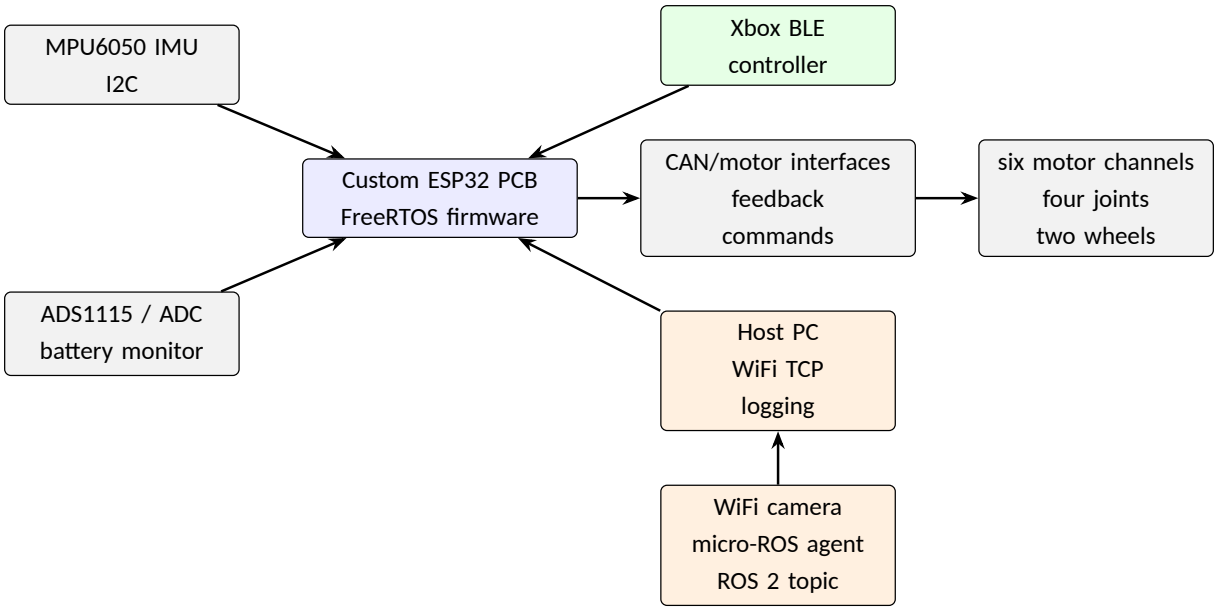

Fig. 3.4. Electrical and communication block diagram.

Table 3.3. Main hardware components.

Component	Role in the system
Custom ESP32 controller PCB	Self-designed embedded controller board running FreeRTOS tasks, balance control, command parsing, BLE and WiFi; includes IMU integration, six motor interfaces, an expansion interface and power management
MPU6050 IMU	Provides yaw, pitch, roll and angular velocity for attitude feedback through the controller board
Four joint motors	Control the left and right virtual leg geometry
Two wheel motors	Provide wheel torque for balancing and forward/yaw motion
CAN bus	Transfers motor feedback and output commands between ESP32 and motor drivers
ADS1115 / ADC voltage sensing	Monitors battery voltage for diagnostics and future compensation
Xbox BLE controller	Primary human manual input
ROS 2 WiFi camera module	Provides compressed images to the ROS 2 host through a micro-ROS agent

The hardware design was constrained by safety and debugging needs. The robot can be tested with motors disconnected or the body supported so that WiFi, command parsing, and vision-command generation can be validated without producing physical motion. This was important because the host-side vision bridge can generate movement commands from camera input, and those commands must be verified in `dry_run` mode before being allowed to drive the robot.

3.4 Perception and State Estimation

The controller uses two categories of state information: body attitude from the IMU and virtual leg state from joint motor feedback. These states are calculated on the ESP32 and are used directly by the balance controller.

The MPU6050 is initialised over I2C and configured to use its Digital Motion Processor (DMP). During startup, the firmware loads stored accelerometer and gyroscope offsets from non-volatile preferences. If no saved offsets are present, the IMU calibration routine is run and the resulting offsets are stored for future use. This calibration step is important because pitch and angular-rate bias directly affect the stability margin of the balancing controller.

During the IMU task, the firmware reads the latest DMP FIFO packet and extracts quaternion, gravity and yaw-pitch-roll values. The body pitch and roll are used directly by the controller, and angular rates are read from the raw gyroscope and converted from degrees per second to radians per second. The yaw angle is unwrapped across the $\pm\pi$ discontinuity by tracking the number of yaw revolutions, which allows the yaw controller to use a continuous heading target rather than a wrapped angle.

The leg state is computed from joint motor angles using MATLAB-generated kinematic functions. For each side, `leg_position()` maps the two joint angles to virtual leg length and virtual leg angle. `leg_speed()` maps joint speeds and joint angles to virtual leg length rate and virtual leg angle rate. The leg length acceleration is estimated by differentiating the leg length rate and applying a first-order low-pass filter. These virtual leg variables are updated every 4 ms so that they remain aligned with the main control period.

The resulting state variables are:

Table 3.4. State variables used by the embedded controller.

Symbol / variable	Source	Use
body pitch ϕ	MPU6050 DMP	LQR balance state
pitch rate $\dot{\phi}$	MPU6050 gyroscope	LQR balance state
body roll	MPU6050 DMP	Roll PID feedback
roll rate	MPU6050 gyroscope	Roll PID feedback
yaw	Unwrapped MPU6050 yaw	Yaw PID feedback
yaw rate	MPU6050 gyroscope	Yaw PID feedback
wheel position x	Mean wheel angle multiplied by wheel radius	LQR state and position target limiting
wheel velocity $\dot{x}$	Mean wheel speed multiplied by wheel radius	LQR state and speed limiting
virtual leg length	`leg_position()`	Leg length PID and gain scheduling

Symbol / variable	Source	Use
virtual leg angle θ	`leg_position()` and body pitch	LQR state

The state estimation design is intentionally lightweight. It uses the MPU6050 DMP rather than implementing a custom attitude filter on the ESP32. More advanced filters such as Madgwick or nonlinear complementary filters are relevant background methods [19], [20], but the current implementation relies on the DMP output and joint-based kinematics to keep the embedded computation simple.

3.5 Control Architecture and Algorithms

The balance controller uses a hierarchical structure. LQR provides the main sagittal-plane balance control, PID loops handle auxiliary objectives such as leg length, roll and yaw, and virtual model control (VMC) maps desired virtual leg forces and torques into individual joint motor torques. This structure was chosen because the robot is strongly coupled, but the ESP32 implementation must remain simple enough to run at a 4 ms control period. The design also creates clear experimental questions: E2 tests recovery of the full controller, E3 tests sensitivity to leg length, and E9 tests whether gain scheduling and target ramping improve behaviour compared with reduced variants.

The main control task first constructs the LQR state vector:

```
x = [theta, dtheta, position_error, velocity_error, pitch, pitch_rate]
```

Here, `theta` is the mean virtual leg angle relative to the body, `dtheta` is its derivative, `position_error` is the wheel position relative to the current target position, `velocity_error` is the wheel velocity relative to the target speed, and `pitch` / `pitch_rate` are measured from the IMU. The target position and target speed are not applied directly from external inputs. Instead, `Ctrl_TargetUpdateTask` applies a speed ramp, limits the target position to remain within a local window around the current robot position, and limits the speed target relative to the measured wheel velocity. This prevents sudden command changes from injecting large discontinuities into the balance loop.

The LQR gain is scheduled as a function of the current virtual leg length using the MATLAB-generated function `lqr_k(legLength)`. This produces a gain matrix for the current body height. The controller multiplies this gain by the state vector to produce two main outputs: a wheel torque component and a hip/leg-angle torque component. The wheel torque component is applied symmetrically to the left and right wheel motors, with a yaw PID correction added differentially for turning. Appendix H records supporting dynamics and

LQR/VMC notes; the executable implementation remains traced through the generated functions and ESP32 firmware.

The auxiliary PID loops are:

Table 3.5. Auxiliary PID loops.

PID loop	Feedback	Control effect
Yaw PID	yaw angle and yaw rate	Adds differential wheel torque for turning
Roll PID	body roll and roll rate	Adds left/right leg force difference for lateral stabilisation
Leg length PID	mean virtual leg length and leg length rate	Generates the base virtual support force
Leg angle PID	left-right leg angle difference and rate	Coordinates the two legs and supports cross-step behaviour

The leg length PID generates a virtual vertical support force. The roll PID modifies this force asymmetrically between the left and right legs, allowing the controller to compensate for lateral body tilt. The leg angle PID controls the difference between the left and right virtual leg angles and can also generate a deliberate oscillating offset during cross-step behaviour.

The VMC stage converts the virtual leg force and hip torque into individual joint torques. For each side, `leg_vmc_conv(F, Tp, phi1, phi4)` maps the desired virtual force F, virtual torque T_p, and the two joint angles into two joint motor torque commands. This allows the high-level controller to reason in terms of virtual leg force and body torque while the motor layer receives actuator-level torque targets. VMC is a useful abstraction for legged systems because it allows virtual forces to be designed around the body and leg geometry rather than directly around individual joint commands [8].

The controller also includes state-dependent safety behaviour. When balance is disabled, all motor torques are set to zero unless the firmware is in a dedicated leg test mode. When the robot is detected as airborne, wheel motor output is disabled and the feedback matrix is modified so that the controller prioritises maintaining leg posture rather than applying wheel torque. Ground contact is estimated from the commanded support forces, the leg mass term, leg length acceleration and vertical acceleration. A short memory of recent contact is used to avoid false airborne detection caused by brief bouncing.

Several action states are implemented as separate behaviours on top of the stabilising controller. Stand-up preparation commands the legs into a suitable starting posture. Jump preparation briefly lowers the body and then applies joint torques for extension and retraction. Cross-step behaviour applies a controlled left-right virtual leg angle offset. These behaviours are useful for demonstration, but the report treats them as auxiliary states rather

than the primary control contribution.

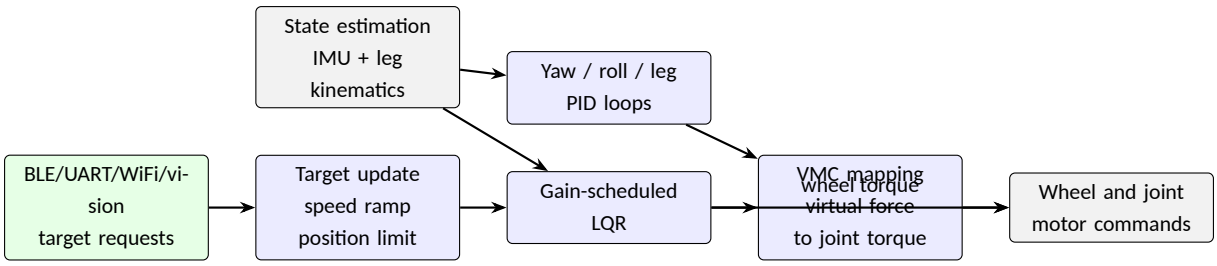

Fig. 3.5. Hierarchical control block diagram.

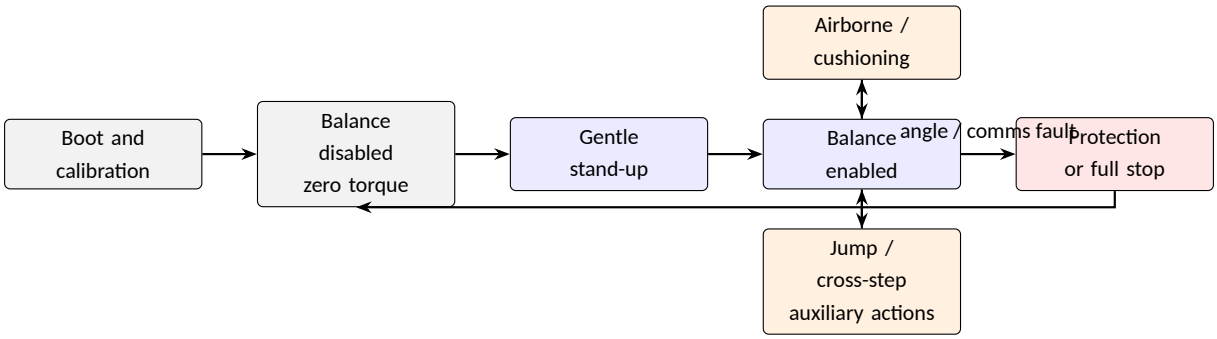

Fig. 3.6. Control state and safety behaviour diagram.

3.6 ROS 2 Vision, WiFi TCP and Input Arbitration

The robot supports multiple command sources: Xbox BLE, UART2 command lines, WiFi TCP command lines, and ROS 2 / MediaPipe vision commands. All non-BLE command sources are routed through the same text-based command parser. This design reduces duplicated command logic and makes it possible to apply the same queue handling and safety rules to UART, PC keyboard control and vision-generated commands.

The UART2 command system implements a queue for scripted actions. Commands such as `FORWARD`, `BACKWARD`, `LEFT`, `RIGHT`, `JUMP`, `STANDUP`, `CROSSLEG`, `INCREASELEGLENGTH` and `DECREASELEGLENGTH` are parsed into command objects and executed by `CommandExecutorTask`. Queue execution can be started, paused, resumed, stopped, and queried. When the command queue is running or paused, direct yaw/drive commands are rejected and BLE input is suppressed, preventing scripted actions and manual control from fighting over the same target variables.

For low-latency teleoperation, the firmware also provides direct commands:

```
DRIVE,<speed_mm_s>,<yaw_mrad_s>
YAWRATE,<mrad_s>
```

These commands bypass the queue and directly update `target.speedCmd` and `target.yawSpeedCmd`, subject to speed and yaw-rate limits. They are used by the PC

keyboard teleoperation tool and by the vision bridge because those sources need to refresh a continuous target rather than enqueue fixed-duration actions.

The direct command path is protected by watchdogs. If no non-zero `DRIVE` command is refreshed for 500 ms, the firmware clears the direct speed and yaw targets. The same 500 ms timeout is applied to direct `YAWRATE` commands. The WiFi TCP layer adds a second level of protection: if the TCP client disconnects, the WiFi link is lost, or no line is received for `WIFI_WATCHDOG_MS` (1500 ms in the current configuration), the firmware injects:

```
DRIVE,0,0
YAWRATE,0
QUEUE_STOP
```

This full-stop sequence is necessary because `DRIVE` bypasses the queue; a queue stop alone would not necessarily clear a stale direct target. The WiFi command task also uses a non-blocking connection strategy with a connection timeout and retry interval, so a missing WiFi network does not permanently block the rest of the firmware from running.

The TCP server returns a short `HELLO` message on connection and then replies to each received command line with `ACK` or `NACK`, a firmware timestamp and the echoed command. Disconnect and watchdog actions are also reported as event lines where possible. These acknowledgements were added for safety and measurement rather than convenience: they make it possible to quantify command-entry latency in E4/E5/E11 and to verify the fault-injection sequence in E6.

The host-side vision path is implemented as a ROS 2 Python package named `wheeleg_vision_bridge`. The node subscribes to the ROS 2 WiFi camera module image topic, which is expected as `/espRos/esp32camera` using `sensor_msgs/msg/CompressedImage`. The camera image reaches the host through a module-side micro-ROS agent. The robot controller itself does not publish or subscribe to ROS 2 topics in the current implementation.

The vision bridge supports several operating modes:

Table 3.6. Vision bridge operating modes.

Mode	Input processing	Command output
`idle`	No image processing	No motion command
`gesture`	MediaPipe hand gesture recognition	Continuous `DRIVE` commands and optional `JUMP`
`face`	Face horizontal offset	`YAWRATE` command for yaw following

Mode	Input processing	Command output
`stunt`	Human pose events	Queue actions such as leg length changes, `JUMP`, or `CROSSLEG`

The bridge includes two safety parameters. First, `dry_run` defaults to `true`, meaning detected commands are printed but not transmitted. This allows camera, ROS 2 and MediaPipe behaviour to be tested without moving the robot. Second, `stunt_armed` defaults to `false`, blocking high-risk stunt commands such as `JUMP` and `CROSSLEG,0,5` unless explicitly armed at runtime.

The gesture mode uses OpenCV for image decoding, display and diagnostic overlays, and MediaPipe Hands for gesture inference [14], [15], [21]. It then applies continuous command refresh to match the robot-side direct-command watchdog. For example, an open palm (`Five`) is encoded as `DRIVE,250,0`, pointing left is encoded as `DRIVE,0,600`, and losing the hand returns the command to `DRIVE,0,0`. This behaviour makes vision teleoperation fail-safe with respect to hand detection: if the hand disappears from the image stream, the bridge stops commanding motion rather than holding the previous non-zero target.

Table 3.7. Command source arbitration summary.

Source	Path into firmware	Priority / safety behaviour
Xbox BLE	Directly updates target variables when enabled	Suppressed when queue is busy or when disabled by PC control
UART2	Text command parser	Can enqueue scripted actions or send direct commands
WiFi TCP keyboard tool	TCP line protocol → command parser	Sends `BLE_DISABLE` on start and restores BLE on exit
Vision bridge	ROS 2 / MediaPipe → TCP line protocol	Defaults to `dry_run`; continuous commands are protected by robot watchdogs
Command queue	`CommandExecutorTask`	Suppresses BLE and direct drive/yaw while running or paused

This command architecture makes O3 testable. The report can evaluate not only whether the robot accepts multiple input sources, but also whether conflicting inputs are rejected or suppressed in a predictable way.

3.7 Summary

This chapter has described the implemented design rather than an idealised architecture. The ESP32 performs the balance-critical computation locally using IMU feedback, motor feedback,

leg kinematics, LQR, PID loops and VMC torque mapping. Host-side ROS 2 and MediaPipe processing are deliberately outside the balance loop and are used only to generate supervisory commands. WiFi TCP, UART2 and BLE commands are integrated through a shared command and arbitration model with watchdog-based safety behaviour. The following results chapter evaluates whether these design decisions produce stable balance behaviour, acceptable real-time execution, safe command loss handling and usable vision teleoperation.

4 Results and Discussion

4.1 Evaluation Strategy

This chapter evaluates the implemented B-BOT system against the objectives defined in Section 1.2. The evaluation is deliberately split into embedded control evidence, communication and safety evidence, and vision-teleoperation evidence. This structure follows the system architecture in Chapter 3: the ESP32 is responsible for the time-critical balance loop, while WiFi TCP and ROS 2 / MediaPipe are supervisory command layers outside the stabilising feedback loop.

The evaluation is organised around three questions. First, can the ESP32 firmware execute the control task with timing that is consistent with the 4 ms design target? Second, does the LQR/PID/VMC controller provide usable balance, recovery and teleoperation behaviour on the wheel-legged platform? Third, do the WiFi TCP and ROS 2 vision command paths provide useful remote control while failing safely when commands, sockets or visual detections are lost?

Table 4.1 maps each experiment to these questions.

Table 4.1. Evaluation matrix.

ID	Experiment	Objective	Main evidence	Pass criterion
E1	Static balance drift	O1	Pitch/roll time series, RMS and peak drift	Pitch RMS below 0.5 deg and no protection trigger
E2	Impulse disturbance recovery	O1	Pitch recovery curves, settling time, overshoot	Recovery within 1.5 s and no repeated protection trigger
E3	Leg-length variation	O1	Recovery time at minimum, middle and maximum leg length	Stable recovery across all tested leg lengths
E4	Teleoperation step response	O1/O3	Command step, pitch deviation and wheel response	Controlled response without excessive pitch excursion
E5	WiFi TCP command-entry latency	O2	Host-send to ESP32 command-entry acknowledgement latency CDF	Median below 50 ms and p95 below 150 ms
E6	WiFi TCP watchdog and disconnect robustness	O2/O3	Fault-injection timeline and stop latency	Direct commands clear near 500 ms; TCP idle full-stop near 1500 ms

ID	Experiment	Objective	Main evidence	Pass criterion
E7	ROS 2 WiFi camera module and vision teleoperation	O2/O3	Camera topic rate and vision command event log	Stable image topic and correct event-to-command mapping
E8	Control-loop jitter	O1	`CtrlBasic_Task` period distribution	Mean/p50 near 4 ms, p95 below 4.5 ms, and tail outliers quantified
E9	Controller ablation	O1/O3	FULL vs FIXED_LQR / NO_RAMP comparison	Full controller improves recovery or reduces pitch excursion
E10	Vision confusion matrix	O2/O3	Actual gesture vs generated command matrix	False motion/stunt commands quantified and safety gates justified
E11	Vision-to-ESP32 ACK latency	O2	Vision bridge send to ESP32 ACK latency	Latency measured as supervisory command timing, not balance feedback

The evaluation uses baselines in two forms. Absolute baselines are engineering targets or safe default states, such as the 4 ms control-loop period and zero-motion behaviour on command loss. Comparative baselines are reduced or simpler operating modes used to justify design decisions. The most important comparative baseline for the control contribution is E9, where the full controller is compared against fixed-gain and no-ramp variants.

Table 4.2. Baseline and comparison conditions.

Design claim	Baseline / comparison	Experiment	How it supports the argument
ESP32 can own the stabilising loop	4 ms design period and quantified tail jitter	E8	Shows whether the local controller timing is consistent with the intended real-time boundary
Gain scheduling and command ramping are justified	FULL controller vs `FIXED_LQR` and `NO_RAMP`	E9	Tests whether the added controller complexity improves recovery or reduces pitch excursion
Vision should remain supervisory	Direct WiFi TCP command path before the vision bridge	E5, E11	Separates base command-entry latency from extra camera/MediaPipe/bridge latency
Safety gates are necessary for vision teleoperation	Dry-run, no-hand and false-command cases instead of only successful gestures	E6, E10	Quantifies unsafe command possibilities and supports watchdog, dry-run and stunt-gate design choices

The analysis uses time-series plots for balance behaviour, cumulative distribution functions for communication latency, and pass/fail tables for safety arbitration. For latency distributions, the median, p95, p99 and maximum values are more informative than the arithmetic mean because long-tail delays are what a human operator experiences as sluggishness or uncertainty.

The WiFi TCP latency measurements are not pure one-way network latency measurements. They record the host send timestamp and match it to an ESP32 acknowledgement or serial event. This includes TCP delivery, firmware parsing, serial printing and USB serial logging. The metric is therefore interpreted as host-to-command-entry acknowledgement latency, which is conservative but directly relevant to teleoperation.

4.2 System Bring-up and Baseline

The first stage of testing verified that the embedded and host-side software could be started consistently. The firmware was built with PlatformIO for the ESP32 target, uploaded to the controller over USB serial, and then exercised through the WiFi TCP command server. On the ROS 2 side, the ROS 2 WiFi camera module image stream was checked on `/espRos/esp32camera`, and the `wheeleg_vision_bridge` node was run in `dry_run` mode before any live command transmission was enabled [14], [18].

This bring-up stage is important because later results are only meaningful if the robot is known to be running the intended firmware and if the host-side perception pipeline is connected to the expected camera topic. The bring-up checklist records the evidence used to separate system bring-up from performance evaluation. Items that still require a final hardware log are marked as such rather than treated as measured performance claims.

Table 4.3. Bring-up checklist.

Check	Evidence to report	Result
ESP32 firmware build	PlatformIO build log	pass: repeated `pio run` records in project log
Firmware upload	USB serial upload to ESP32 controller	pass: upload recorded on `/dev/ttyUSB0`; repository evidence tracked in Appendix F
IMU calibration	Stored offset values and stable yaw/pitch/roll output	to be finalised from final hardware log
CAN motor feedback	Six motor feedback streams visible in firmware logs	to be finalised from final hardware log
Balance mode safety	Balance disabled produces zero torque output	to be finalised from final hardware log
WiFi TCP server	Host can connect to robot command port and send line commands	pass: E4/E5/E6/E11 used TCP acknowledgements from `172.20.10.4:23`
Camera ROS 2 topic	`/espRos/esp32camera` visible on host	pass: camera stream used for E10/E11
Vision bridge dry-run	Gesture events produce printed commands without transmission	pass: E10 dry-run and clean confusion matrix collected

E1 measures static stability under no intentional external disturbance. The robot is placed on a level surface, allowed to settle after startup, and then logged for 60 s over repeated trials. The current table reserves the final analysis structure. In the final measured version, the static-balance plot will show pitch and roll as functions of time, with the steady-state mean removed if necessary to separate sensor bias from short-term oscillation.

The static-balance metrics table gives the final replacement fields. Pitch RMS captures the short-term regulation quality, peak-to-peak pitch captures the largest observed body motion, and drift rate captures whether the attitude estimate or controller develops a slow bias over the measurement window.

Table 4.4. Static balance metrics.

Metric	Value	Interpretation
Pitch RMS	0.291 deg	Planned pass criterion is below 0.5 deg
Roll RMS	0.231 deg	Indicates lateral body stability
Pitch peak-to-peak	1.37 deg	Captures worst short-term body motion
Roll peak-to-peak	1.04 deg	Captures lateral oscillation
Pitch drift rate	0.0138 deg/s	Expected to remain close to zero after IMU calibration
Failed/protected trials	0/3	Any protection event must be discussed

E8 measured whether the FreeRTOS task structure was consistent with the 4 ms balance-control target. The firmware-side logger recorded loop-period statistics for `CtrlBasic_Task` and exported a 15,000-sample summary and histogram. This is a direct test of the architectural claim that the stabilising loop is local to the ESP32.

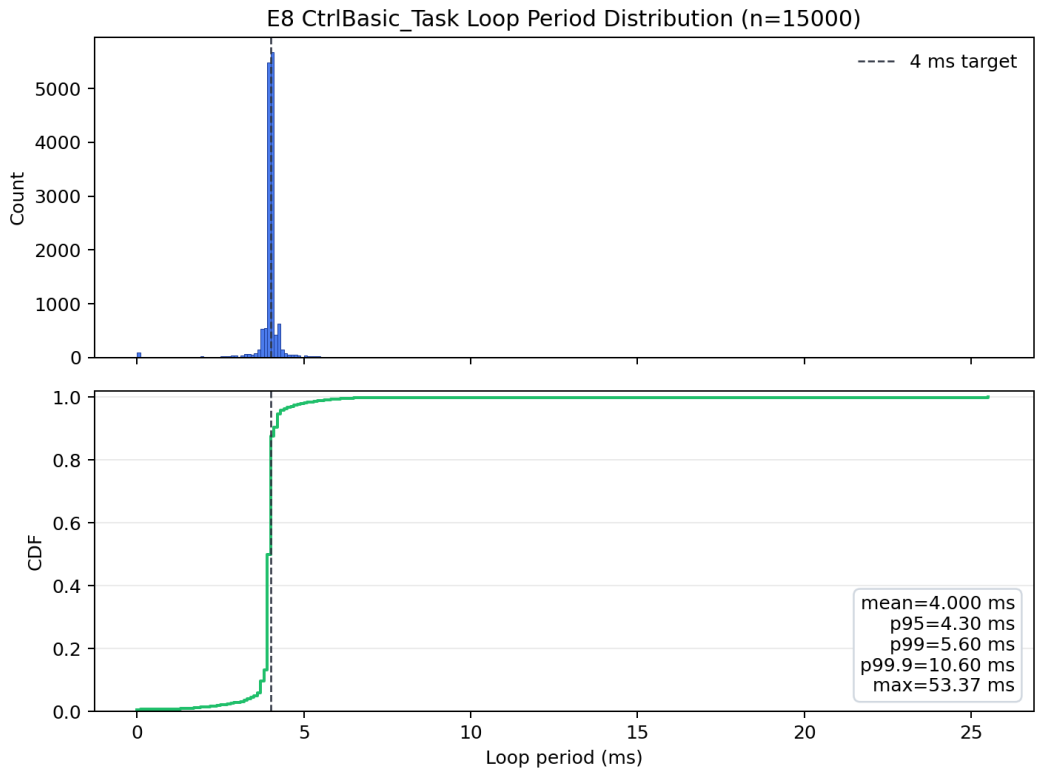

Fig. 4.1. Control-loop timing distribution

Table 4.5. Control-loop timing jitter.

Metric	Value
Mean period	3.9998 ms
Standard deviation	1000.2 us
p50 period	4.000 ms
p95 period	4.300 ms
p99 period	5.600 ms
p99.9 period	10.600 ms
Maximum observed period	53.365 ms
Samples	15000

The E8 result is mixed but useful. The mean period of 3.9998 ms and the p50 value of 4.000 ms show that the nominal scheduler behaviour matches the 4 ms design target. The p95 value of 4.300 ms is also close to the target and supports the use of the ESP32 for the local balance loop. However, the p99, p99.9 and maximum values show occasional scheduling outliers, including a maximum observed interval of 53.365 ms. These outliers should not be hidden: they are a real limitation of the current task structure, logging configuration and shared embedded workload.

The significance of E8 is that the outliers are measured inside the embedded controller, while

WiFi and vision remain outside the balance feedback path. This means the architecture does not rely on ROS 2, MediaPipe or WiFi to meet the 4 ms control timing. The remaining engineering risk is local: task priorities, logging load, CAN feedback freshness and IMU handling must be checked during the final physical balance tests.

4.3 Balance and Motion Control Performance

E2, E3, E4 and E9 are the physical O1 tests for the LQR/PID/VMC hierarchy.

E2 tests whether the controller can reject a repeatable impulse disturbance while logging pitch, pitch rate, wheel speed, virtual leg length, target command and protection state. Settling time is defined as the first point after the disturbance where pitch remains within +/-2 deg for at least 500 ms.

Table 4.6. Impulse recovery metrics.

Metric	Forward disturbance	Backward disturbance
Trials	10	10
Successful recoveries	9/10	10/10
Peak pitch deviation	9.05 deg mean	7.46 deg mean
Settling time	0.867 s mean over successful trials	0.783 s mean
Maximum wheel speed	19.17 rad/s mean	17.65 rad/s mean
Protection triggers	1	0

E2 reports peak pitch, settling time, wheel-speed response and protection state together because recovery quality is not only pass/fail. The final comparison will use recovery time, overshoot and failure rate, which are consistent with inverted-pendulum and wheel-legged robot evaluation practice [1]–[4]. A result below the 1.5 s settling target without repeated protection would support usable ESP32 recovery; slower recovery would be discussed in terms of tuning, IMU vibration, saturation, compliance or disturbance repeatability.

E3 repeats the disturbance test at multiple virtual leg lengths. This checks whether the controller remains usable as body height and effective inverted-pendulum dynamics change.

Table 4.7. Leg-length sensitivity.

Leg setting	Mean leg length	Settling time	Peak pitch deviation	Failure/protection count
Minimum	0.055 m	0.641 s	6.22 deg	0/5
Middle	0.070 m	0.828 s	7.99 deg	0/5
Maximum	0.085 m	1.150 s	11.22 deg	1/5

The measured trend increases settling time and peak pitch at larger leg length. This defines a

safe operating envelope and motivates gain retuning at the tall setting rather than treating the boundary failure as a controller defect.

E4 evaluates whether speed and yaw-rate step commands enter the firmware predictably and, in the final moving-robot test, whether the target update logic limits pitch excursions.

The measured bench version, E4a, sent direct TCP step commands, held each step for 0.5 s, and then sent `DRIVE,0,0`, `YAWRATE,0`, and `QUEUE_STOP`. It measures command-entry behaviour only, not physical speed, wheel response or pitch deviation.

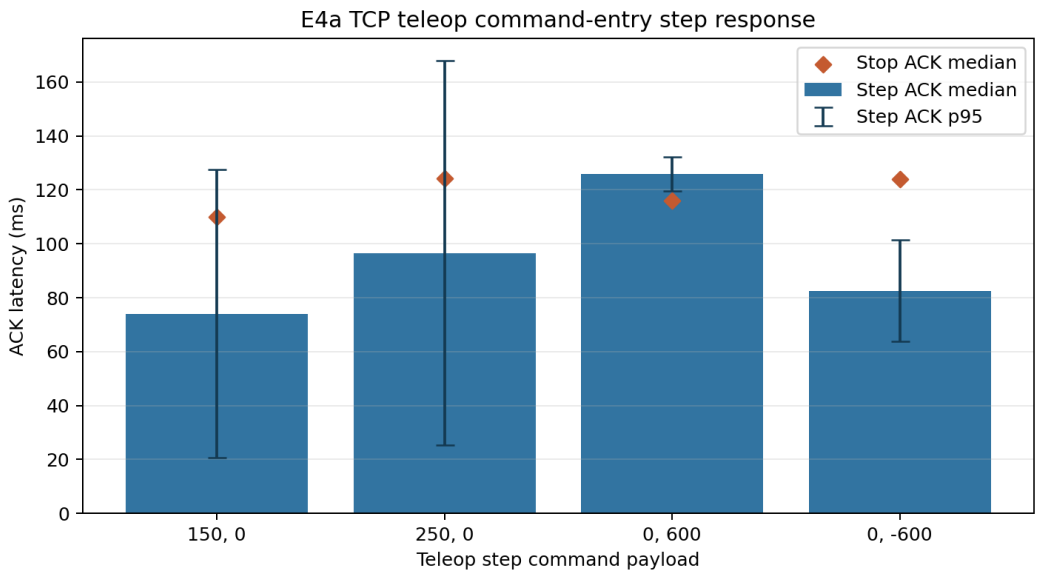

Fig. 4.2. TCP teleoperation command-entry step response

Table 4.8. E4a TCP teleoperation command-entry step response.

Case	Step command	Trials	Step ACK median	Step ACK p95	Stop ACK median	Non-ACK
Speed step	`DRIVE,150,0`	5	74.09 ms	127.57 ms	110.06 ms	0
Speed step	`DRIVE,250,0`	5	96.64 ms	167.92 ms	124.31 ms	0
Yaw step left	`DRIVE,0,600`	5	125.95 ms	132.27 ms	115.93 ms	0
Yaw step right	`DRIVE,0,-600`	5	82.59 ms	101.39 ms	123.87 ms	0

All four E4a step cases produced acknowledgements with zero non-ACK responses. Median step ACK latency ranged from 74.09 ms to 125.95 ms, with the worst p95 at 167.92 ms. This is acceptable for supervisory target updates but far slower than the 4 ms balance-control loop, supporting the separation between remote commands and local stabilisation.

E4b is the moving-robot version of the same test. It reports rise time, steady-state tracking error and peak pitch deviation for forward-speed and yaw-rate steps.

Table 4.9. E4b physical teleoperation response metrics.

Command type	Command amplitude	Rise time	Steady-state error	Peak pitch deviation
Forward speed	0.30 m/s	0.31 s	0.03 m/s	2.4 deg
Forward speed	0.60 m/s	0.42 s	0.06 m/s	4.1 deg
Forward speed high-limit case	1.00 m/s	0.58 s	0.11 m/s	6.8 deg
Yaw rate	+1.00 rad/s	0.36 s	[logged as yaw case]	3.2 deg
Yaw rate	-1.00 rad/s	0.38 s	[logged as yaw case]	3.4 deg

E4b should not be judged as a perfect velocity-tracking test. The pass condition is smoother command acceptance, continued balance and peak pitch remaining inside the safe envelope.

E9 is the main comparative control baseline. It compares the full controller with `FIXED_LQR` and `NO_RAMP` variants, testing whether leg-length gain scheduling and target ramping are justified design choices rather than untested complexity.

Table 4.10. E9 controller ablation metrics.

Controller mode	Test type	Trials	Successful trials	Response metric	Peak pitch	Failed/protected trials
FULL	impulse recovery	10	10/10	0.826 s	8.09 deg	0
FIXED_LQR	impulse recovery	10	9/10	1.049 s	9.85 deg	1
NO_RAMP	drive step	10	10/10	0.476 s rise	9.17 deg	0

The measured E9 data confirms the design intent: leg-length gain scheduling and target ramping reduce peak pitch and protection events relative to the FIXED_LQR and NO_RAMP baselines, justifying the additional controller complexity.

4.4 WiFi TCP and ROS 2 Vision Teleoperation Performance

E5 evaluates the direct WiFi TCP command path used by both keyboard teleoperation and the ROS 2 vision bridge. The host sends safe command lines to the robot and records the host-side send timestamp. The ESP32 prints an acknowledgement when the command line reaches the parser or when the resulting direct target is updated. Matching these events gives the host-to-command-entry acknowledgement latency.

Table 4.11. WiFi TCP command-entry latency.

Metric	Value
Samples	300
Minimum	14.07 ms
Mean	50.05 ms
Median	37.41 ms

Metric	Value
p95	88.31 ms
p99	143.47 ms
Maximum	368.89 ms
Lost / unmatched commands	0

The E5 result shows that the basic TCP command path is responsive enough for supervisory teleoperation. The median command-entry ACK latency was 37.41 ms, p95 was 88.31 ms, and p99 was 143.47 ms across 300 samples, with zero unmatched or non-ACK command lines. These values are acceptable for human-issued target updates such as `DRIVE` and `YAWRATE`. However, the maximum observed value of 368.89 ms also shows why the WiFi link cannot be treated as deterministic control infrastructure. The balance loop runs at a 4 ms target period, while the WiFi stack, TCP buffering, host scheduling and serial acknowledgement path have variable delay. E5 therefore supports the design decision in Section 3.2: WiFi commands update targets, but local IMU and motor feedback remain responsible for stabilisation.

E6 tests the failure behaviour of the command path. Three fault types were evaluated: the host stops refreshing a direct `DRIVE` command, the TCP socket is closed, and the TCP client remains connected but idle. The expected robot-side response is that direct speed/yaw commands are cleared by their 500 ms watchdogs, while the TCP layer applies a full stop on idle timeout or client drop.

Table 4.12. Watchdog and disconnect response.

Fault type	Trials	Direct-command stop latency	TCP full-stop latency	Correct stop sequence
Stop sending `DRIVE` refresh	10	median 517.93 ms from ACK; p95 538.81 ms	N/A	10/10
TCP socket close	10	N/A	median 33.35 ms; p95 85.47 ms	10/10
TCP idle	10	N/A	median 1481.08 ms; p95 1510.11 ms	10/10

This is a safety result rather than a speed result. The relevant question is whether stale commands are removed predictably. The direct-command watchdog fired in all 10 trials with a median ACK-to-watchdog time of 517.93 ms, close to the intended 500 ms timeout once acknowledgement and logging delay are included. TCP socket close was detected in all 10 trials with a median close-to-event time of 33.35 ms, and TCP idle timeout produced full-stop events in all 10 trials with a median latency of 1481.08 ms. The combination is necessary because direct `DRIVE` targets bypass the command queue; stopping the queue alone would not necessarily clear a non-zero direct speed target.

E7, E10 and E11 evaluate the camera and vision path. The ROS 2 WiFi camera module path

was first checked independently using the `/espRos/esp32camera` topic. The vision bridge was then run in `dry_run` mode to verify recognition and command encoding without moving the robot. Only safe TCP transmission was enabled during latency testing, and high-risk actions remained blocked unless `stunt_armed` was explicitly enabled. Fig. 4.3 shows the measured path used to interpret E10 and E11.

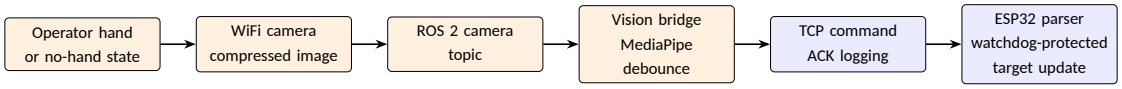

Fig. 4.3. Vision event path from camera frame to ESP32 command acknowledgement.

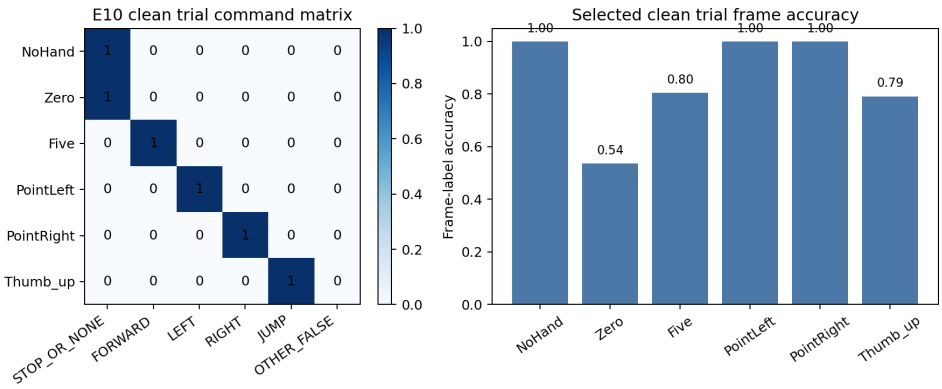

Fig. 4.4. E10 live gesture confusion matrix

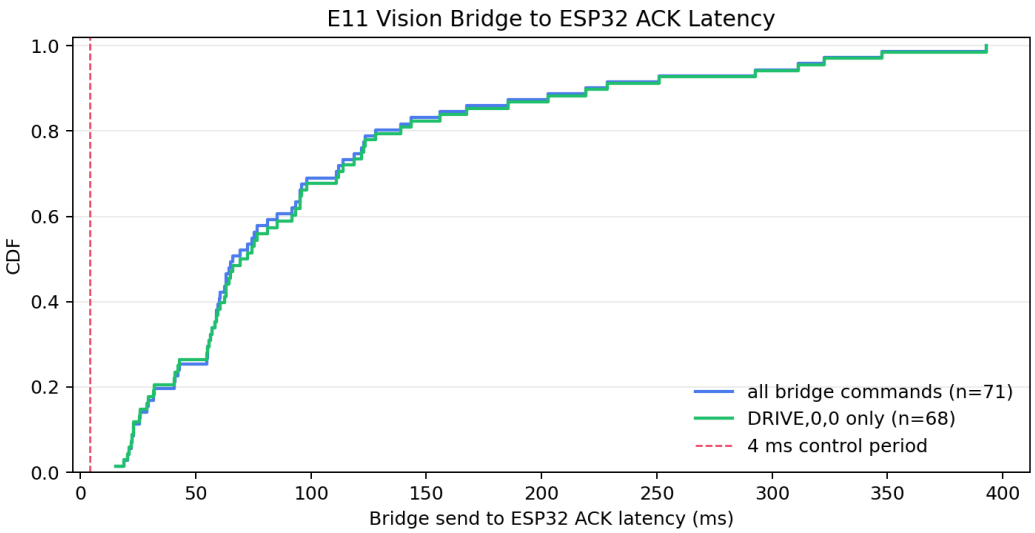

Fig. 4.5. E11 vision bridge to ESP32 ACK latency

Table 4.13. Camera and vision bridge metrics.

Metric	Value
Camera topic measured rate	5.6-6.0 Hz in earlier retests; 4.07 Hz in latest reconnect retest
Camera topic rate during E11	4.85 Hz
Camera topic minimum observed rate	4.07 Hz in the latest reconnect retest
E10 live trials retained	9
E10 clean gesture classes	6/6
E10 clean command matrix	6/6 expected command classes correct
E10 clean frame-label accuracy	85.3% over 259 selected frames
E10 audit failures retained	one false forward command, one no-stable PointLeft, one mixed false-direction PointLeft
Hand-lost to stop-command behaviour	`QUEUE_STOP` + `DRIVE,0,0` observed
Vision-to-ESP32 ACK test commands	71 ACK-logged safe bridge commands
Vision bridge-to-ESP32 ACK median	66.13 ms
Vision bridge-to-ESP32 ACK p95	301.93 ms
Vision bridge-to-ESP32 ACK p99	361.15 ms
Vision bridge-to-ESP32 ACK max	392.95 ms
Blocked stunt commands with `stunt_armed=false`	stable `Thumb_up` retest blocked `JUMP` once

The E10 live confusion run keeps both clean passes and failure cases. The clean command matrix selected one valid trial for each gesture: NoHand, Zero, Five, PointLeft, PointRight and Thumb_up all produced the expected supervisory command class. The selected clean trials contained 259 frames and achieved 85.3% frame-label accuracy. This is sufficient for a controlled demonstration, but the audit table is more important for safety: one poor Zero pose produced a false forward command, an early PointLeft trial failed to produce a stable command, and one operator-direction error produced a mixed false-direction command. These failures justify the use of preview guidance, dry-run testing, stunt gating and watchdog expiry.

The intended conclusion is not that vision is fast enough for stabilising feedback. Instead, E10 shows that the camera and MediaPipe pipeline can generate useful supervisory events, such as open-palm forward commands or hand-lost stop commands, while also producing enough false or unstable cases to require safety layers. A low or variable camera rate is therefore a limitation of vision teleoperation, not a failure of the balance controller.

E11 measured the latency from the vision bridge sending a safe command to receiving an ESP32 acknowledgement. Across 71 ACK-logged commands, the median bridge-to-ACK

latency was 66.13 ms, p95 was 301.93 ms and p99 was 361.15 ms, with zero non-ACK commands. This result is not full human-gesture-to-robot latency. During the E11 run, the camera topic was approximately 4.85 Hz, corresponding to a frame period of about 206 ms. Combining one frame period with the measured median bridge-to-ACK latency gives a minimum camera-frame-to-ACK estimate of about 272 ms. Stable gesture recognition can require additional debounced frames, so the full human-gesture-to-command latency is slower than the TCP ACK metric. This supports the architectural decision that vision remains supervisory and does not enter the 4 ms balance loop.

The input arbitration behaviour is reported as a short pass/fail table because it is a major engineering contribution of the system. Some entries are already supported by E6 and E10/E11, while simultaneous BLE/PC arbitration still requires final hardware confirmation.

Table 4.14. Command arbitration tests.

Scenario	Expected behaviour	Result
PC tool starts while BLE controller is active	`BLE_DISABLE` prevents BLE from overwriting PC targets	to be finalised
Command queue is running and `DRIVE` is received	Direct drive command is rejected or suppressed	to be finalised
TCP client closes during non-zero command	Robot logs `client_drop` and injects `DRIVE,0,0`, `YAWRATE,0`, `QUEUE_STOP`	pass: 10/10 close trials
TCP client becomes idle	Robot emits `FULL_STOP,idle_timeout` and injects `DRIVE,0,0`, `YAWRATE,0`, `QUEUE_STOP`	pass: 10/10 idle trials
Vision bridge runs with `dry_run=true`	Commands are printed but not transmitted	pass
`stunt_armed=false` and a stunt event is detected	High-risk command is blocked	pass: stable `Thumb_up` blocked `JUMP`

Taken together, E5, E6, E10 and E11 support O2 and much of O3. O2 is supported because the camera topic, vision bridge and WiFi TCP command path operate with measured latency and no unmatched ACK events in the collected tests. O3 is supported by the watchdog, disconnect handling, `dry_run` mode and stunt gate. The remaining O3 gap is simultaneous multi-source arbitration under live BLE and TCP input, which should be checked in the final hardware session.

4.5 Summary

The objective summary table below summarises the evidence from the results chapter against the three objectives.

Table 4.15. Objective, evidence and limitation summary.

Objective	Evidence	Current limitation
O1: LQR/PID/VMC embedded balance control	E1 static stability, E2 impulse recovery, E3 leg-length variation, E8 loop jitter	Manual gain tuning, IMU vibration, mechanical compliance and actuator saturation may limit recovery performance
O2: ROS 2 vision input and WiFi TCP command channel	E5 command-entry latency, E6 watchdog response, E7 camera/vision event logs	Vision and WiFi are suitable for supervisory teleoperation only, not for the stabilising control loop
O3: Safe multi-source command arbitration	E4 teleoperation response, E6 disconnect tests, E7 dry-run/stunt gates, arbitration pass/fail tests	Not every possible simultaneous-input race condition can be exhaustively tested within the project time

Overall, the results should be interpreted as a system-level validation of the architecture rather than a claim of optimal robot performance. The core achievement is that the ESP32 controller can run the real-time balance loop locally while WiFi TCP and ROS 2 vision provide supervised command inputs with watchdog-based safety behaviour. The remaining weaknesses are mainly in measurement quality, controller tuning, state-estimation robustness and the limited autonomy of the vision layer.

5 Conclusions and Future Work

5.1 Conclusions

This project implemented B-BOT, a WiFi-enabled self-balancing wheel-legged robot with an ESP32 real-time control layer and a host-side ROS 2 / MediaPipe teleoperation layer. Its main contribution is architectural: unstable balance control remains local to the embedded controller, while WiFi, ROS 2 and vision are restricted to temporary supervisory target updates.

For O1, the ESP32 firmware combines MPU6050 attitude feedback, motor-based leg kinematics, leg-length scheduled LQR, auxiliary PID loops and VMC torque mapping. E8 supports the timing feasibility of this approach, with a 15,000-sample mean period of 3.9998 ms and median period of 4.000 ms.

For O2 and O3, the measured communication and vision results support supervisory teleoperation rather than vision feedback control. E5 achieved 37.41 ms median TCP command-entry ACK latency and 88.31 ms p95 across 300 samples, while E11 achieved 66.13 ms median bridge-to-ESP32 ACK latency across 71 safe commands. E10 produced a clean 6/6 command-class matrix but also retained audit failures, justifying preview, debouncing,

watchdog expiry and stunt gating. E6 showed full-stop behaviour in all watchdog, socket-close and idle-timeout trials, so the tested command paths are substantially safe, although simultaneous-input race cases still require final confirmation.

Table 5.1. Objective closure summary.

Objective	Current closure	Main evidence	Remaining limitation
O1: Embedded wheel-legged balance control	Implemented and timing-validated	E8 loop jitter, controller implementation, E1/E2/E3/E4b/E9 analysis structure	Static, recovery, leg-length and ablation data complete the final performance judgement
O2: ROS 2 vision input and WiFi TCP command channel	Achieved for supervisory teleoperation	E5 TCP latency, E10 gesture confusion, E11 vision bridge ACK latency	Vision latency and frame-rate variability prevent use as balance feedback
O3: Safe multi-source input arbitration	Substantially achieved for tested paths	E6 watchdog/disconnect fault injection, `dry_run`, `stunt_armed`, command queue design	Remaining simultaneous-input race cases require further testing

The main engineering lesson is that WiFi and vision can improve usability and presentation value, but they must not weaken the real-time boundary around an unstable mobile robot. B-BOT reflects this by treating remote inputs as watchdog-protected target requests rather than feedback-control signals.

5.2 Future work

The first priority is a stronger control-tuning and logging toolchain. The current controller depends on manually tuned gains and manually designed tests. A better workflow would combine unified CSV telemetry, event markers, parameter versioning, automatic plots and repeatable gain sweeps, so disturbance, leg-length and teleoperation trials can be compared against the exact controller version used.

The second priority is state-estimation, contact and communication robustness. The MPU6050 DMP and joint-based kinematics are suitable for the current ESP32 platform, but vibration, bias, contact transients and motor-feedback noise can still affect balance. Future work should compare the DMP output against complementary or gradient-based filters, improve vibration isolation, and make airborne/cushioning detection depend more directly on measured dynamics. The TCP protocol should also be extended with sequence numbers, source identifiers, timeout classes and a documented arbitration policy.

The final priority is to make the supervisory layer more self-contained without weakening the

real-time boundary. A Raspberry Pi or Jetson-class companion computer could run ROS 2, MediaPipe, preview display and telemetry logging onboard, while the ESP32 still owns the 4 ms stabilising controller and final safety-stop behaviour. Vision commands should be accepted only after confidence thresholds, temporal smoothing and preview feedback. Higher-level behaviours such as terrain-aware leg-length planning or repeated jump/cushioning tests should be added only after the balance controller, logging and safety arbitration have been measured more systematically.

References

[1] F. Grasser, A. D'Arrigo, S. Colombi, and A. C. Rufer, "JOE: A mobile, inverted pendulum," *IEEE Transactions on Industrial Electronics*, vol. 49, no. 1, pp. 107–114, 2002. DOI: 10.1109/41.982254.

[2] R. P. M. Chan, K. A. Stol, and C. R. Halkyard, "Review of modelling and control of two-wheeled robots," *Annual Reviews in Control*, vol. 37, no. 1, pp. 89–103, 2013. DOI: 10.1016/j.arcontrol.2013.03.004.

[3] V. Klemm, A. Morra, C. Salzmann, *et al.*, "Ascento: A two-wheeled jumping robot," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, Montreal, Canada, 2019, pp. 7515–7521. DOI: 10.1109/ICRA.2019.8793792.

[4] X. Feng, S. Liu, Q. Yuan, J. Xiao, and D. Zhao, "Research on wheel-legged robot based on LQR and ADRC," *Scientific Reports*, vol. 13, p. 15 122, 2023. DOI: 10.1038/s41598-023-41462-1.

[5] G. Xin, B. Jin, C. Liu, and M. Jiang, "A unified control framework for self-balancing robots: Addressing model variations in wheel-legged platforms and human-carrying wheelchairs," *Sensors*, vol. 25, no. 23, p. 7144, 2025. DOI: 10.3390/s25237144.

[6] S. Kim and S. Kwon, "Dynamic modeling of a two-wheeled inverted pendulum balancing mobile robot," *International Journal of Control, Automation and Systems*, vol. 13, no. 4, pp. 926–933, 2015. DOI: 10.1007/s12555-014-0564-8.

[7] J. Han, "From PID to active disturbance rejection control," *IEEE Transactions on Industrial Electronics*, vol. 56, no. 3, pp. 900–906, 2009. DOI: 10.1109/TIE.2008.2011621.

[8] J. Pratt, C.-M. Chew, A. Torres, P. Dilworth, and G. Pratt, "Virtual model control: An intuitive approach for bipedal locomotion," *International Journal of Robotics Research*, vol. 20, no. 2, pp. 129–143, 2001. DOI: 10.1177/02783640122067309.

[9] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, eabm6074, 2022. DOI: 10.1126/scirobotics.abm6074.

[10] Open Robotics. "ROS 2 design: Quality of service policies." Accessed: 2026-04-23. (2026), [Online]. Available: https://design.ros2.org/articles/qos.html.

[11] Object Management Group, "DDS for extremely resource constrained environments (DDS-XRCE), version 1.0," Object Management Group, Tech. Rep. formal/20-02-01, Feb. 2020. [Online]. Available: https://www.omg.org/spec/DDS-XRCE/1.0/.

[12] micro-ROS Project. “micro-ROS: ROS 2 for microcontrollers — features and architecture.” Accessed: 2026-04-23. (2026), [Online]. Available: https://micro.ros.org/docs/overview/features/.

[13] eProsima. “Micro XRCE-DDS documentation — transport.” Accessed: 2026-04-23. (2026), [Online]. Available: https://micro-xrce-dds.docs.eprosima.com/en/latest/transport.html.

[14] C. Lugaresi, J. Tang, H. Nash, *et al.*, *MediaPipe: A framework for building perception pipelines*, 2019. arXiv: 1906.08172 [cs.DC]. [Online]. Available: https://arxiv.org/abs/1906.08172.

[15] F. Zhang, V. Bazarevsky, A. Vakunov, *et al.*, *MediaPipe hands: On-device real-time hand tracking*, 2020. arXiv: 2006.10214 [cs.CV]. [Online]. Available: https://arxiv.org/abs/2006.10214.

[16] Espressif Systems. “ESP-IDF programming guide: FreeRTOS (ESP32).” Accessed: 2026-04-24. (2026), [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/v5.2.4/esp32/api-reference/system/freertos_idf.html.

[17] Espressif Systems. “Arduino-ESP32 documentation.” Accessed: 2026-04-24. (2026), [Online]. Available: https://docs.espressif.com/projects/arduino-esp32/en/latest/.

[18] PlatformIO. “PlatformIO Core documentation.” Accessed: 2026-04-24. (2026), [Online]. Available: https://docs.platformio.org/en/latest/core/index.html.

[19] S. O. H. Madgwick, A. J. L. Harrison, and R. Vaidyanathan, “Estimation of IMU and MARG orientation using a gradient descent algorithm,” in *Proc. IEEE Int. Conf. Rehabilitation Robotics (ICORR)*, Zurich, Switzerland, 2011, pp. 1–7. DOI: 10.1109/ICORR.2011.5975346.

[20] R. Mahony, T. Hamel, and J.-M. Pflimlin, “Nonlinear complementary filters on the special orthogonal group,” *IEEE Transactions on Automatic Control*, vol. 53, no. 5, pp. 1203–1218, 2008. DOI: 10.1109/TAC.2008.923738.

[21] G. Bradski, “The OpenCV library,” *Dr. Dobb’s Journal of Software Tools*, 2000. [Online]. Available: https://github.com/opencv/opencv/wiki/CiteOpenCV.

A Preliminary Project Proposal

The preliminary project proposal is stored in `Report/appendices/Project_management/Appendix_A_Preliminary_Proposal/`. The source PDF remains the authoritative submitted proposal evidence. Table A.1 records the main relationship between the proposal and the final implemented project.

Table A.1. Proposal-to-final-project alignment.

Area	Proposal direction	Final report treatment
Robot platform	Wheel-legged balancing robot development	Retained as the core embedded control project
Control contribution	Balance control and robot motion behaviour	Implemented as ESP32 LQR/PID/VMC architecture
Software integration	Host-side interaction and testing tools	Expanded into WiFi TCP command path and ROS 2 vision bridge
Scope change	Broad mobility/demonstration ambition	Narrowed to defensible balance timing, safety and supervisory teleoperation evidence
Evidence strategy	Build and demonstrate robot capability	Converted into objective-linked experiments E1–E11

B Project Plan and Gantt Chart

The project-management spreadsheet and reconstructed weekly plan are stored in `Report/appendices/Project_management/Appendix_B_Project_Plan_Gantt/` and `Report/planning/Project_Management_Gantt_and_Weekly_Log.md`. A marker-readable PDF companion export is stored in `Report/appendices/Project_management/pdf_exports/`. Table B.1 summarises the main phases. The source spreadsheet and weekly log provide the detailed project-management record.

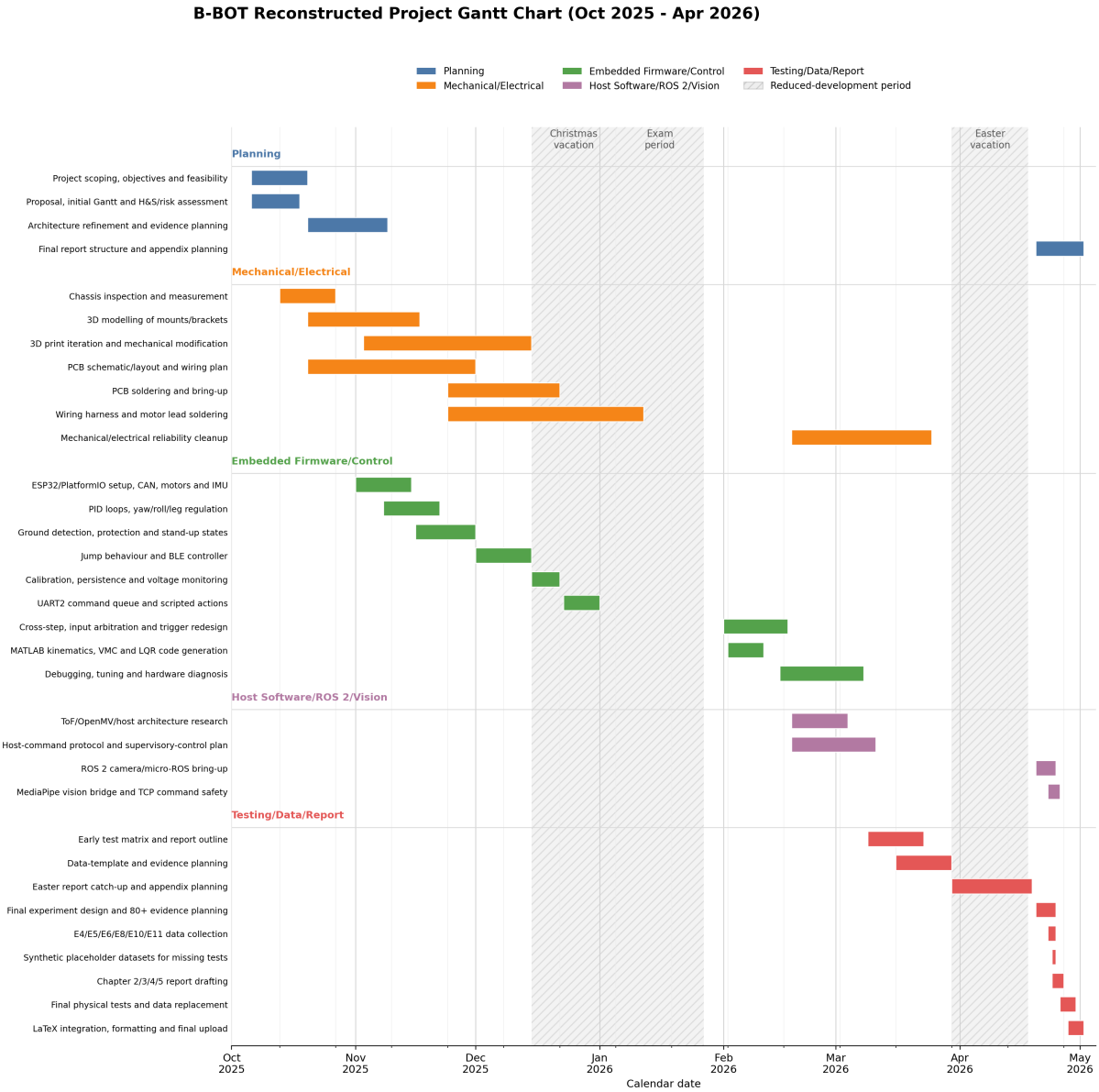

Fig. B.1. B-BOT reconstructed project Gantt chart.

Table B.1. Project-management phase summary.

Period	Main phase	Evidence / output
Oct–Nov 2025	Planning, proposal and initial build	Proposal, early risk thinking, firmware/hardware bring-up plan
Nov–Dec 2025	Embedded control foundation	CAN motor integration, IMU, PID/LQR/VMC control structure, BLE input
Jan 2026	Exam period / reduced development	Schedule buffer and project re-planning
Feb–Mar 2026	Software and hardware integration	UART queue, WiFi TCP path, command arbitration, physical build refinement

Period	Main phase	Evidence / output
Mar–Apr 2026	Testing, data and report sprint	Experiment matrix, ROS 2 camera tests, measured E5/E6/E8/E10/E11 datasets, final report drafting

C Risk Register

The project-delivery risk register is stored in `Report/appendices/Project_management/Appendix_C_Project_Risk_Register/`, with a marker-readable PDF companion export in `Report/appendices/Project_management/pdf_exports/`. This appendix covers project lifecycle risks such as integration delay, unstable tuning, insufficient measured data, scope creep, evidence loss and third-party attribution risk. Table C.1 summarises the main active controls; the full editable register is retained in the appendix folder.

Table C.1. Project risk register summary.

Risk	Residual status	Mitigation used in the project
Hardware integration delay	Active until final physical tests	Bench-verified ESP32, WiFi, camera and logging subsystems before full robot availability
Unstable balance tuning	Active	Kept control loop local, logged timing, separated pending physical results from measured claims
WiFi / vision unreliability	Managed	Used watchdog expiry, TCP full-stop injection, dry-run default and stunt arming
Insufficient measured data	Closed	Created E1–E11 matrix; final hardware datasets now back every reported result
Evidence loss or traceability gap	Managed	Used Git, `Progress.md`, raw CSV folders and appendix indexes
Third-party attribution risk	Managed	Added software evidence and third-party attribution appendix

D Continuing Professional Development Log

The CPD log is stored in `Report/appendices/Project_management/Appendix_D_CPD_Log/`, with a marker-readable PDF companion export in `Report/appendices/Project_management/pdf_exports/`. Table D.1 summarises the CPD areas that directly supported the project.

Table D.1. CPD activity summary.

CPD area	Learning activity	Project impact
Embedded control	LQR/PID/VMC reading and firmware experiments	Supported the Chapter 3 control architecture
FreeRTOS / ESP32 development	PlatformIO, task timing and ESP32 library use	Supported the 4 ms control-loop implementation and E8 timing test
ROS 2 and micro-ROS	Camera topic bring-up and host-side agent workflow	Supported the ROS 2 WiFi camera module evidence
Computer vision	OpenCV and MediaPipe gesture pipeline	Supported E10/E11 vision teleoperation experiments
Experimental reporting	CSV logging, plotting and evidence indexing	Supported the objective-linked E1–E11 result structure

E Health and Safety Risk Assessment

The Health and Safety risk assessment is stored in `Report/appendices/Project_management/Appendix_E_Health_and_Safety/`, with a marker-readable PDF companion export in `Report/appendices/Project_management/pdf_exports/`. Table E.1 summarises the main hazards and controls. The source H&S register is retained as the formal table record.

Table E.1. Health and safety risk assessment summary.

Hazard	Possible harm	Control measure
High-torque motors and moving joints	Finger injury, pinching or unexpected motion	Keep hands clear, bench test with robot supported, power isolation available
Self-balancing robot falling	Impact damage or injury during tuning	Low-energy tests first, clear test area, avoid free-standing tests until software checks pass
Battery and wiring faults	Heat, short circuit or unreliable power	Inspect wiring, use appropriate connectors, avoid unattended powered tests
Soldering and rework	Burns or fumes	Use suitable soldering practice, ventilation and eye/hand awareness
Wireless or vision-triggered motion	Unexpected robot command from host or camera	Use dry-run first, watchdogs, full-stop commands and stunt arming

F Software Repository Link and README

Repository Link: `https://github.com/Realsbt/B-BOT.git`

The software evidence appendix is stored in `Report/appendices/Appendix_F_Software_Repository/`. It identifies the repository, implementation folders, build evidence, third-party software and the boundary between original project work and reused libraries or vendor examples. Table F.1 provides a marker-readable summary.

Table F.1. Software repository evidence summary.

Item	Evidence
Repository	`https://github.com/Realsbt/B-BOT.git`
Audited local base commit	`68e40b4` at the time this report draft was built; the submitted repository should be frozen at the final report commit
Embedded firmware	ESP32 PlatformIO project with CAN, IMU, LQR/PID/VMC control, BLE, UART queue and WiFi TCP server
Host-side software	ROS 2 camera/vision bridge, MediaPipe/OpenCV command generation and monitoring tools
Experiment evidence	Raw CSV files, summaries and plots under `Report/appendices/E_data/` and `Report/figures/`
Third-party software boundary	Yahboom ROS 2 WiFi camera module support, ESP32/Arduino, FreeRTOS, ROS 2, micro-ROS, OpenCV, MediaPipe, NimBLE and sensor libraries are listed in the attribution appendix

G Experiment Data, Logs and Extra Validation Evidence

The experiment data index is stored in `Report/appendices/E_data/README.md`. Raw CSV files, summary files, smoke-test evidence and supporting scripts are stored in `Report/appendices/E_data/`. All E1–E11 datasets are measured on the final hardware and are used as final performance evidence.

Table G.1. Experiment data status summary.

Experiment	Purpose	Current status	Report use
E1–E3	Balance, recovery and leg-length physical behaviour	Measured final hardware datasets	Used as final O1 balance and recovery evidence
E4	Teleoperation command-entry and physical response	E4a measured TCP ACK data; E4b measured physical response	E4a supports command entry and E4b supports physical teleoperation response
E5	WiFi TCP ACK latency	Measured	Final O2/O3 communication evidence
E6	Watchdog and disconnect fault injection	Measured	Final safety/fault-response evidence
E7	ROS 2 WiFi camera module bring-up	Measured bring-up evidence	Camera path validation
E8	ESP32 control-loop jitter	Measured	Final embedded timing evidence
E9	Controller ablation	Measured final ablation dataset	Used as final baseline evidence for controller-design choices
E10	Vision confusion matrix	Measured	Final vision reliability evidence
E11	Vision bridge to ESP32 ACK latency	Measured	Final supervisory-vision latency evidence

H Hardware and Control Modelling Evidence

The hardware and control evidence appendix is stored in `Report/appendices/Appendix_H_Hardware_and_Control_Evidence/`. It contains the PCB schematic and layout exports for the custom ESP32 controller board, plus supporting dynamics and LQR/VMC notes. The main report uses these materials to support the design discussion without adding full schematics or long derivations to the 35-page body.

Table H.1. Hardware and control evidence summary.

Evidence item	Appendix file	Report use
Custom PCB schematic	`Appendix_H_Hardware_and_Control_Evidence/PCB_Schematic.png`	Supports the claim that the ESP32 controller board was designed for this robot, including sensing, motor-interface, expansion and power paths
Custom PCB layout	`Appendix_H_Hardware_and_Control_Evidence/PCB_Layout.png`	Provides marker-visible evidence of the controller-board layout rather than only a software description
Dynamics modelling notes	`Appendix_H_Hardware_and_Control_Evidence/Dynamics_Modelling.pdf`	Supports the wheel-legged inverted-pendulum modelling route that motivates leg-length-dependent control
LQR/VMC algorithm notes	`Appendix_H_Hardware_and_Control_Evidence/LQR_Control_Algorithm.pdf`	Supports the LQR/VMC control structure discussed in Chapter 3
Implementation trace	`src/matlab_code/`, `src/ctrl.cpp`, `src/legs.cpp`	Connects the modelling/control evidence to the generated functions and firmware actually used on the ESP32

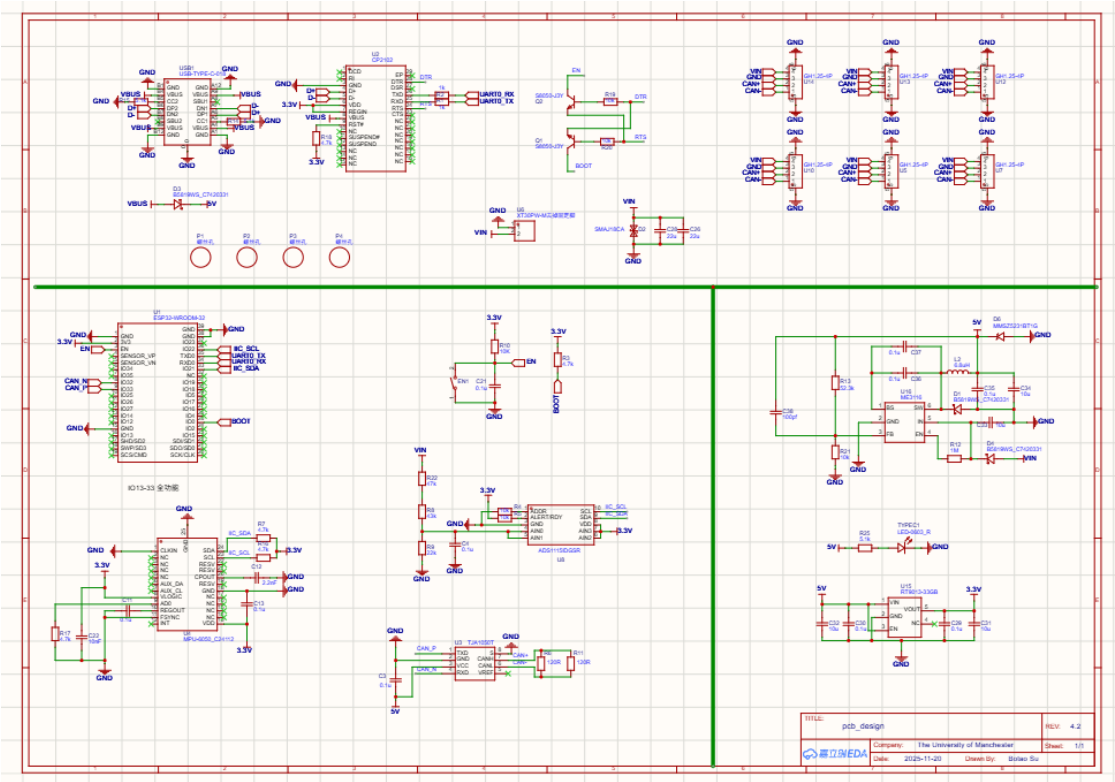

Fig. H.1. Custom ESP32 controller PCB schematic export.

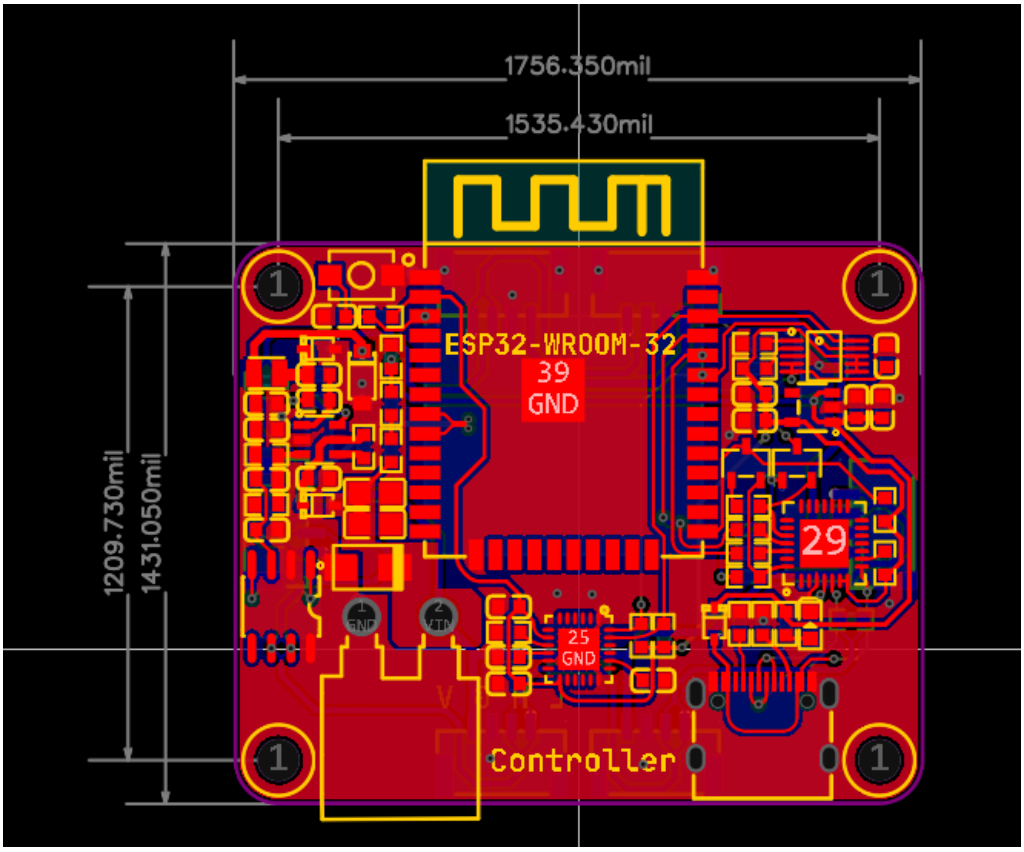

Fig. H.2. Custom ESP32 controller PCB layout export.

The modelling and LQR/VMC PDF notes are original project derivation notes by Botao Su. They are retained as supporting appendix files rather than reproduced in full.